

پایتون پلاس



پایتون پلاس

1404

عماد رضا عباسپور

مجموعه تکنو پلاس

خراسان رضوی، مشهد

09050944755

مقدمه

سلام عرض میکنم خدمت همه شما، امیدوارم که حالتون خوب باشه و آماده باشد برای یادگیری پایتون، من توی این کتاب سعی کردم از کلمات و مباحث سخت پرهیز کنم تا یادگیری برای همه سنین راحت باشد این کتاب مخصوص نوجوانان، جوانان و علاقه مندان به رشته تکنولوژی و برنامه نویسی تالیف شده است. امیدوارم که بتونید از این کتاب و مطالبش نهایت استفاده و بهره را ببرید.

اهداف

1. آموزش زبان برنامه نویسی پایتون به زبان ساده و مناسب برای همه سنین
2. ارتقا سطح علمی دانش آموزان/دانشجویان
3. ایجاد ایده و علاقه در علاقه مندان به برنامه نویسی
4. پرورش دانش آموزان دارای استعداد در حوزه تکنولوژی و برنامه نویسی

مشخصات

- **پایتون پلاس: مسیر کامل یادگیری پایتون از مقدماتی تا پیشرفته**
- [عماد رضا عباس پور]
- فارسی
- پایه‌ای برای دوره‌های پیشرفته برنامه‌نویسی و علوم داده
- شامل متن آموزشی، مثال‌های کدنویسی، تمرین‌ها و پروژه‌های عملی
- آموزشی و کاربردی
- مناسب یادگیری گام به گام پایتون
- مناسب برای مبتدیان و نوجوانان علاقه‌مند به برنامه‌نویسی و دانشجویان و افراد خودآموز
- شامل مفاهیم پایه، کتابخانه‌های استاندارد، پروژه‌های عملی، برنامه‌نویسی شی گرا و مسیرهای پیشرفته
- یادگیری گام به گام با مثال‌های عملی
- مناسب برای تمام سنین و سطوح
- با استفاده از مستندات رسمی پایتون
- ترکیب تئوری و عمل

ویژگی های کتاب

۱. آموزش مرحله به مرحله از مقدماتی تا پیشرفته

کتاب «پایتون پلاس» با رویکرد گام به گام نوشته شده است. خواننده از مفاهیم پایه پایتون مثل متغیرها، حلقه ها و توابع آغاز می کند و به تدریج با مفاهیم پیشرفته مثل ماژول ها، کتابخانه ها، پروژه های واقعی و حتی اصول برنامه نویسی شیءگرا آشنا می شود.

مزیت: مناسب برای مبتدیان و افرادی که می خواهند مسیر یادگیری منسجم داشته باشند.

۱۱. آموزش عملی با پروژه های واقعی

هر فصل شامل پروژه های عملی است؛ از ماشین حساب و دفترچه یادداشت ساده گرفته تا بازی حدس عدد و پروژه های تحلیل داده. این پروژه ها مهارت های عملی خواننده را تقویت می کنند و یادگیری صرفاً نظری نیست.

مزیت: با انجام پروژه ها، خواننده اعتماد به نفس پیدا می کند و مفاهیم در ذهنش تثبیت می شوند.

۱۱۱. معرفی کتابخانه ها و ابزارهای کاربردی

کتاب «پایتون پلاس» علاوه بر آموزش زبان، به معرفی کتابخانه های محبوب پایتون مانند **Turtle**، **Pygame**، **Matplotlib** و **Tkinter**، **NumPy**، **Pandas** می پردازد و نحوه استفاده عملی از آن ها را نشان می دهد.

مزیت: یادگیری سریع ابزارهای کاربردی باعث می شود خواننده بتواند پروژه های واقعی بسازد.

۱۱۱۱. نکات و مسیرهای پیشرفت بعدی

در انتهای کتاب، مسیرهای پیشرفت برای خواننده مشخص شده است: از پروژه های متوسط و پیشرفته گرفته تا ورود به یادگیری ماشین، هوش مصنوعی و وب اپلیکیشن. نکات انگیزشی و توصیه های عملی به خواننده کمک می کند مسیر یادگیری خود را ادامه دهد.

مزیت: کتاب فقط محدود به مفاهیم پایه نیست و مسیر حرفه ای شدن را هم نشان می دهد.

۱۱۱۱۱. تمرین ها و چالش های متعدد

در هر فصل، تمرین ها و چالش های متنوع ارائه شده است که خواننده را به حل مسئله و تفکر تحلیلی دعوت می کند.

مزیت: این تمرین ها باعث می شوند مفاهیم عمیق تر در ذهن خواننده جا بیفتند و مهارت حل مسئله افزایش یابد.

۱۱۱۱۱۱. طراحی برای همه سنین و سطوح

کتاب «پایتون پلاس» به گونه ای نوشته شده که حتی نوجوانان و افراد تازه کار نیز می توانند با آن پیش بروند، ولی برای افراد پیشرفته نیز نکات حرفه ای و مسیرهای پیشرفته ارائه شده است.

مزیت: مخاطب گسترده ای دارد و برای مدرسه، دانشگاه و یادگیری شخصی مناسب است.

« فهرست »

« فهرست مطالب »

بخش ۱: شروع مسیر برنامه‌نویسی

برنامه‌نویسی یعنی چه؟
آشنایی با پایتون و نصب آن
اجرای اولین برنامه: "سلام دنیا!"
استفاده از محیط‌های کدنویسی (VS Code و IDLE)

بخش ۲: پایه‌های زبان پایتون

متغیرها و انواع داده
عملگرها و محاسبات
ورودی و خروجی داده‌ها
دستورات شرطی (if, else, elif)
حلقه‌ها (while و for)

بخش ۳: ساختارهای داده

لیست‌ها (List)
تاپل‌ها (Tuple)
دیکشنری‌ها (Dictionary)
مجموعه‌ها (Set)

بخش ۴: توابع و ماژول‌ها

تعریف و فراخوانی توابع
پارامترها و مقدار بازگشتی
ماژول‌ها و کتابخانه‌ها
ساخت ماژول شخصی

بخش ۵: کار با فایل‌ها و پروژه‌های کوچک

خواندن و نوشتن فایل‌ها

پروژه‌های پایتون (از مطالب یادگرفته شده)

پروژه: دفترچه یادداشت متنی ساده

پروژه: ماشین حساب ساده

پروژه: بازی حدس عدد

بخش ۶: گام بعدی در مسیر یادگیری

معرفی کتابخانه‌های محبوب پایتون (مثل Turtle و Pygame)

نکات و مسیرهای پیشرفت بعدی

درس اول

درس اول

برنامه‌نویسی یعنی چه؟

برنامه‌نویسی پایتون یعنی نوشتن دستوراتی به زبان پایتون برای انجام کارهای مختلف در کامپیوتر. پایتون (Python) یکی از محبوب‌ترین زبان‌های برنامه‌نویسی در دنیاست که به خاطر سادگی، خوانایی و قدرت زیادش، هم بین مبتدی‌ها و هم برنامه‌نویسان حرفه‌ای بسیار محبوب است.

برای اینکه بهتر بفهمیم، بیایید قدم به قدم توضیح دهیم:

برنامه‌نویسی چیست؟

برنامه‌نویسی یعنی گفت‌وگو با کامپیوتر به زبانی که آن بفهمد. ما با نوشتن مجموعه‌ای از دستورها (کدها)، به کامپیوتر می‌گوییم چه کاری انجام دهد؛ مثلاً جمع دو عدد را حساب کند، یک تصویر را نشان دهد، یا حتی یک بازی را اجرا کند.

پایتون چیست؟

پایتون یک زبان برنامه‌نویسی سطح بالا (High-Level) است، یعنی ما در آن به جای درگیر شدن با جزئیات سخت‌افزار، فقط روی منطق و هدف برنامه تمرکز می‌کنیم. این زبان در سال ۱۹۹۱ توسط «گیدو فان روسوم» ساخته شد و امروزه یکی از مهم‌ترین زبان‌ها در جهان است.

چرا به آن می‌گویند سطح بالا؟

چون شبیه زبان انسان است و دستورهایش بسیار ساده و قابل فهم‌اند. مثلاً در زبان‌های قدیمی‌تر مثل C باید از علامت‌ها و ساختارهای پیچیده استفاده می‌کردی، ولی در پایتون خیلی روان و ساده می‌نویسی.

یک مثال ساده از کد پایتون:

```
name = input("اسم شما چیست؟")  
print("سلام", name, "آمدی")
```

این چند خط ساده باعث می‌شود کامپیوتر اسم شما را بپرسد و سپس با شما خوش‌آمد بگوید!

کاربردهای پایتون:

پایتون تقریباً در همه زمینه‌های فناوری استفاده می‌شود:

طراحی وب (مثلاً با Flask و Django)

هوش مصنوعی و یادگیری ماشین

تحلیل داده‌ها و آمار

ساخت بازی

خودکارسازی کارهای تکراری

برنامه‌نویسی ربات‌ها

ساخت نرم‌افزارهای دسکتاپ و موبایل

مزایای پایتون:

یادگیری بسیار آسان

جامعه کاربری بزرگ

وجود هزاران کتابخانه آماده برای هر کار

متن‌باز و رایگان بودن

قابل اجرا روی ویندوز، لینوکس، مک و حتی موبایل

چرا پایتون برای مبتدی‌ها عالی است؟

چون نیاز به نوشتن کد زیاد ندارد و به راحتی می‌توان نتیجه کار را دید. مثلاً برای چاپ یک جمله کافی است بنویسی:

```
print("سلام دنیا")
```

در حالی که در زبان‌های دیگر ممکن است ۵ تا ۱۰ خط بنویسی.

کتابخانه‌ها (Libraries):

کتابخانه‌ها در پایتون مثل جعبه ابزار هستند. مثلاً اگر بخواهی نمودار بکشی، از کتابخانه Matplotlib استفاده می‌کنی. برای هوش مصنوعی از TensorFlow یا PyTorch، و برای ساخت بازی از Pygame.

محیط اجرای پایتون:

برای نوشتن برنامه‌های پایتون می‌توانی از نرم‌افزارهایی مثل:

IDLE (نصب‌شده همراه خود پایتون)

VS Code

PyCharm

Jupyter Notebook

استفاده کنی.

پایتون چگونه کار می‌کند؟

کد پایتون توسط «مفسر (Interpreter)» اجرا می‌شود. یعنی خط‌به‌خط کد را می‌خواند و اجرا می‌کند. برخلاف بعضی زبان‌ها که اول باید «کامپایل» شوند (تبدیل کامل به زبان ماشین)، پایتون بلافاصله اجرا می‌شود و همین کار را سریع‌تر و راحت‌تر می‌کند.

پایتون در دنیای واقعی:

شرکت‌های بزرگی مثل گوگل، یوتیوب، نتفلیکس، اینستاگرام و اسپاتیفای از پایتون استفاده می‌کنند. مثلاً الگوریتم پیشنهاد فیلم در نتفلیکس با پایتون نوشته شده است.

یادگیری پایتون چه فایده‌ای دارد؟

اگر پایتون یاد بگیری، می‌توانی:

شغل برنامه‌نویسی بگیری

پروژه‌های شخصی انجام دهی

وبسایت بسازی

ربات تلگرام طراحی کنی

یا حتی بازی بسازی!

نکته مهم:

پایتون فقط یک زبان نیست؛ بلکه پلی است برای ورود به دنیای برنامه‌نویسی، منطق و تفکر الگوریتمی. یعنی به تو یاد می‌دهد چطور برای هر مسئله راه‌حل منطقی طراحی کنی.

نمونه دیگر:

```
for i in range(1, 6):  
    print("عدد:", i)
```

این برنامه اعداد ۱ تا ۵ را چاپ می‌کند. به همین سادگی می‌توان حلقه‌ها و تکرار را یاد گرفت.

جمع‌بندی:

برنامه‌نویسی پایتون یعنی استفاده از زبانی ساده، قدرتمند و همه‌فن‌حریف برای ساخت هر نوع برنامه، از ساده‌ترین تا پیچیده‌ترین. پایتون به تو اجازه می‌دهد خلاق باشی و هر ایده‌ای را به واقعیت تبدیل کنی.

درس دوم

درس دوم

آشنایی با پایتون و نصب آن

پایتون (Python) یکی از محبوب‌ترین زبان‌های برنامه‌نویسی دنیاست که برای اهداف گوناگونی مثل ساخت وب‌سایت، تحلیل داده، هوش مصنوعی، خودکارسازی کارها و حتی ساخت بازی استفاده می‌شود. این زبان به خاطر سادگی و شباهت زیاد به زبان انسان، برای مبتدی‌ها بسیار مناسب است.

برای شروع کار با پایتون، باید آن را روی سیستم خود نصب کنی. نصب آن بسیار آسان است و در کمتر از چند دقیقه انجام می‌شود.

ابتدا وارد سایت رسمی پایتون به نشانی python.org

شو و در بخش Downloads، نسخه مخصوص سیستم عامل خودت (ویندوز، لینوکس یا مک) را دانلود کن. پس از دانلود، فایل نصب را اجرا کن و در همان ابتدای صفحه، گزینه‌ی "Add Python to PATH" را تیک بزن تا پایتون در مسیر سیستم ثبت شود.

بعد روی Install Now کلیک کن تا مراحل نصب به‌صورت خودکار انجام شود.

پس از پایان نصب، برای اطمینان از درست بودن نصب، در قسمت Command Prompt (CMD) بنویس:

```
python --version
```

اگر شماره نسخه پایتون را دیدی، یعنی با موفقیت نصب شده است.

همراه پایتون، برنامه‌ای به نام IDLE هم نصب می‌شود که محیط ساده‌ای برای نوشتن و اجرای کدهاست. البته بسیاری از برنامه‌نویسان حرفه‌ای از محیط‌هایی مثل VS Code یا PyCharm استفاده می‌کنند چون امکانات بیشتری دارند.

پایتون کاملاً رایگان و متن‌باز است، یعنی هرکسی می‌تواند از آن استفاده کند و حتی آن را گسترش دهد. بعد از نصب، می‌توانی اولین برنامه خودت را با این دستور بنویسی:

```
print("سلام دنیا")
```

با همین چند گام ساده، وارد دنیای جذاب و بی‌نهایت برنامه‌نویسی با پایتون می‌شوی.

درس سوم

درس سوم

اجرای اولین برنامه: "سلام دنیا!"

وقتی پایتون را نصب کردی، وقت آن است که اولین برنامه‌ات را اجرا کنی. اولین برنامه معمولاً ساده‌ترین است و فقط یک جمله روی صفحه نمایش می‌دهد. در پایتون، برای نمایش یک پیام از دستور `print()` استفاده می‌شود.

برای شروع، برنامه IDLE را که همراه پایتون نصب شده اجرا کن. در پنجره‌ای که باز می‌شود، خطی شبیه به این می‌بینی:

<<<

این یعنی پایتون آماده‌ی دریافت دستور توست.

حالا بنویس:

```
print("سلام دنیا!")
```

و دکمه Enter را بزن.

بلافاصله می‌بینی که جمله‌ی «سلام دنیا!» روی صفحه چاپ می‌شود.

این اولین برنامه‌ی تو در پایتون است و نشان می‌دهد که محیط به‌درستی کار می‌کند. اگر خواستی همین کد را در یک فایل جدا ذخیره کنی، از منوی `File → New File` استفاده کن. کد را بنویس و سپس آن را با نامی مثل `hello.py` ذخیره کن. بعد از ذخیره، دکمه `F5` را بزن تا برنامه اجرا شود.

به همین سادگی، اولین گام در مسیر برنامه‌نویسی با پایتون را برداشتی!

درس چهارم

درس چهارم

استفاده از محیط‌های کدنویسی (VS Code و IDLE)

وقتی پایتون را نصب می‌کنی، همراه آن یک محیط ساده به نام IDLE هم نصب می‌شود. IDLE در واقع محیط پیش‌فرض پایتون است و برای شروع یادگیری بهترین گزینه محسوب می‌شود. نام آن از عبارت Integrated Development and Learning Environment گرفته شده است.

در IDLE می‌توانی به صورت مستقیم کد بنویسی، اجرا کنی و نتیجه را فوراً ببینی. وقتی برنامه را باز می‌کنی، یک پنجره با علامت سه فلش (<<<) می‌بینی که به آن Shell می‌گویند. در Shell می‌توانی دستورها را خط به خط بنویسی و همان لحظه اجرا کنی.

مثلاً اگر بنویسی:

```
print("سلام پایتون")
```

و Enter بزنی، همان لحظه خروجی روی صفحه نشان داده می‌شود. این ویژگی برای تمرین و تست کدهای کوچک بسیار مفید است.

اما وقتی بخواهی پروژه‌های بزرگ‌تری بنویسی، نیاز به محیطی پیشرفته‌تر داری. در اینجا نرم‌افزاری مثل Visual Studio Code (VS Code) وارد می‌شود. VS Code یکی از بهترین و پرمفردترین محیط‌های برنامه‌نویسی در جهان است. این نرم‌افزار رایگان است و توسط شرکت مایکروسافت ساخته شده است.

در VS Code می‌توانی چندین فایل را با هم باز کنی، کدها را رنگ‌بندی شده ببینی و راحت‌تر اشکال‌یابی کنی. با نصب افزونه‌ی رسمی «Python Extension» می‌توانی پایتون را به طور کامل در VS Code اجرا کنی. این افزونه امکاناتی مثل اجرای سریع برنامه، تشخیص خطاها، پیشنهاد خودکار دستورات و حتی اجرای خط‌به‌خط (Debug) را فراهم می‌کند.

برای نصب آن کافی است بعد از نصب VS Code، وارد بخش Extensions شوی و عبارت "Python" را جستجو کنی.

سپس روی افزونه‌ی منتشرشده توسط مایکروسافت کلیک و آن را نصب کن.

بعد از نصب، هر فایلی با پسوند py را می‌توانی مستقیماً در VS Code باز و اجرا کنی.

مزیت دیگر VS Code این است که می‌توانی ظاهر آن را شخصی‌سازی کنی. مثلاً تم تاریک (Dark Theme) فعال کنی، فونت‌ها را تغییر دهی یا پلاگین‌های کمکی نصب کنی. همچنین امکان اجرای ترمینال (Command Line) درون خود نرم‌افزار وجود دارد، بدون نیاز به باز کردن پنجره جداگانه.

در مقایسه، IDLE سبک‌تر و ساده‌تر است و برای شروع یادگیری و تمرین مناسب‌تر می‌باشد. VS Code اما برای پروژه‌های حرفه‌ای، چند فایل و طولانی‌گزینه‌ی بهتری است. در نتیجه پیشنهاد می‌شود ابتدا با IDLE شروع کنی تا با اصول پایتون آشنا شوی. وقتی دستت راه افتاد، به VSCode مهاجرت کن تا تجربه‌ای حرفه‌ای‌تر از برنامه‌نویسی داشته باشی.

هر دو محیط هدفشان این است که کدنویسی را راحت‌تر، سریع‌تر و جذاب‌تر کنند. اما انتخاب نهایی به نوع پروژه و سطح مهارت تو بستگی دارد. اگر تازه‌کار هستی، IDLE ساده‌ترین مسیر است؛ ولی اگر می‌خواهی یک برنامه‌نویس واقعی و حرفه‌ای شوی، VS Code ابزار اصلی تو خواهد بود.

درس پنجم

درس پنجم

متغیرها و انواع داده

در برنامه‌نویسی با پایتون، متغیر (Variable) یکی از پایه‌ای‌ترین و مهم‌ترین مفاهیم است. متغیر در واقع یک محفظه‌ی نام‌دار است که مقداری را در حافظه‌ی کامپیوتر نگه می‌دارد. به زبان ساده، متغیر مثل یک «برچسب» روی یک داده است تا بتوانیم آن را بعداً دوباره استفاده کنیم.

مثلاً وقتی می‌نویسی:

```
x = 5
```

یعنی عدد ۵ را در متغیری به نام x ذخیره کرده‌ای. حالا هر وقت از x استفاده کنی، پایتون به جای آن مقدار ۵ را در نظر می‌گیرد.

در پایتون نیازی نیست نوع داده (Data Type) را از قبل مشخص کنی. برخلاف بعضی زبان‌ها مثل C یا Java که باید بنویسی `int x = 5`؛ در پایتون فقط کافی است بنویسی `x = 5`. خود پایتون به طور خودکار تشخیص می‌دهد که مقدار عددی است، رشته‌ای است، یا نوع دیگر.

■ انواع داده‌های اصلی در پایتون:

اعداد صحیح (int):

برای ذخیره‌ی اعداد بدون اعشار استفاده می‌شود.
مثلاً:

```
age = 15
```

```
year = 2025
```

اعداد اعشاری (float):

برای ذخیره‌ی عددهایی با قسمت اعشاری.

```
pi = 3.14
```

```
temperature = 27.5
```

رشته‌ها (str):

رشته یا "String" یعنی متنی از کاراکترها که درون کوتیشن قرار می‌گیرد.

```
name = "فاطمه"
```

```
greeting = 'سلام دنیا!'
```

می‌توانی از هر دو نوع کوتیشن ' ' یا " " استفاده کنی.

بولی (bool):

برای بیان درست یا نادرست بودن چیزی استفاده می‌شود. فقط دو مقدار دارد:

```
is_sunny = True
```

```
is_raining = False
```

لیست‌ها (list):

لیست یعنی مجموعه‌ای از داده‌ها که می‌تواند از هر نوع باشد و قابل تغییر است.

```
numbers = [1, 2, 3, 4, 5]
```

```
names = ["علی", "زهرا", "مهدی"]
```

می‌توانی با اندیس (index) به عناصر لیست دسترسی پیدا کنی:

```
print(names[0]) # چاپ "علی"
```

تاپل‌ها (tuple):

مثل لیست هستند ولی قابل تغییر نیستند (یعنی نمی‌توانی مقدارشان را عوض کنی).

```
point = (10, 20)
```

دیکشنری‌ها (dict):

داده‌ها را به صورت "کلید-مقدار" ذخیره می‌کنند، شبیه دفترچه تلفن.

```
student = {"name": "امیر", "age": 14, "grade": 9}
```

```
print(student["name"]) # خروجی: امیر
```

مجموعه‌ها (set):

مجموعه‌ها مثل لیست هستند ولی تکراری‌ها را حذف می‌کنند.

```
colors = {"red", "blue", "green", "red"}
```

```
print(colors) # است "red" خروجی شامل فقط یک
```

نام‌گذاری متغیرها در پایتون: 

فقط می‌توانند شامل حروف، اعداد و زیرخط (_) باشند.

نمی‌توانند با عدد شروع شوند.

نباید از کلمات رزرو شده‌ی پایتون (مثل for, if, class) استفاده کرد.

نام‌ها باید معنی‌دار باشند تا کد خواناتر شود.

مثلاً:

✓ درست:

```
user_name = "Ali"
```

```
age = 15
```

✗ نادرست:

```
2name = "Ali"
```

```
for = 10
```

■ تغییر نوع داده‌ها (Type Casting):

گاهی لازم است نوع داده را تغییر دهی.
پایتون برای این کار توابعی دارد:

```
x = "5"
```

```
y = int(x) # تبدیل رشته به عدد
```

```
z = str(10) # تبدیل عدد به رشته
```

```
a = float(7) # تبدیل عدد صحیح به اعشاری
```

■ بررسی نوع داده:

می‌توانی با تابع `type()` نوع داده را ببینی:

```
x = 3.14
```

```
print(type(x)) # خروجی: <class 'float'>
```

■ جمع‌بندی:

♦ متغیرها برای ذخیره و استفاده از داده‌ها به کار می‌روند.

- ♦ نوع داده تعیین می‌کند که چه نوع اطلاعاتی در متغیر ذخیره شده است.
- ♦ پایتون به صورت خودکار نوع داده را تشخیص می‌دهد، اما می‌توانی آن را تغییر دهی.
- ♦ دانستن انواع داده یکی از اولین و مهم‌ترین گام‌ها در برنامه‌نویسی پایتون است.

درس ششم

درس ششم

عملگرها و محاسبات

در پایتون، عملگرها (Operators) ابزارهایی هستند که برای انجام محاسبات، مقایسه‌ها و کارهای منطقی استفاده می‌شوند.

هر عملگر بر روی داده‌ها یا متغیرها (که به آن‌ها عملوند - Operands گفته می‌شود) کار می‌کند. مثلاً در عبارت $a + b$ ، $+$ یک عملگر است و a و b عملوند هستند.

پایتون انواع مختلفی از عملگرها دارد که هر کدام کار مخصوصی انجام می‌دهند.

در ادامه، با مهم‌ترین دسته‌های عملگرها آشنا می‌شویم 📌

1. عملگرهای حسابی (Arithmetic Operators)

این عملگرها برای انجام محاسبات ریاضی استفاده می‌شوند:

عملگر	توضیح	مثال	خروجی
$+$	جمع	$3 + 5$	8
$-$	تفریق	$4 - 10$	6
$*$	ضرب	$6 * 2$	12
$/$	تقسیم اعشاری	$2 / 9$	4.5
$//$	تقسیم صحیح	$2 // 9$	4
$\%$	باقی مانده تقسیم	$3 \% 10$	1
$**$	توان	$2 ** 2$	4

مثلاً به کد زیر توجه کنید:

```
a = 7
b = 2
print(a + b) # 9
print(a ** b) # 49
print(a % b) # 1
```

2. عملگرهای مقایسه‌ای (Comparison Operators)

برای مقایسه‌ی دو مقدار و برگرداندن نتیجه‌ی درست (True) یا نادرست (False) استفاده می‌شوند:

عملگر	توضیح	مثال	خروجی
==	برابر است با	5 == 5	True
!=	نابرابر است با	4 != 3	True
<	بزرگ‌تر است از	6 < 10	True
>	کوچک‌تر است از	5 > 8	False
<=	بزرگ‌تر یا مساوی	7 <= 7	True
>=	کوچک‌تر یا مساوی	3 >= 2	True

مثال:

```
x = 10
y = 20
print(x < y) # True
print(x == y) # False
```

3. عملگرهای منطقی (Logical Operators)

نکته: برای ترکیب چند شرط یا عبارت بولی (درست/غلط) استفاده می‌شوند.

عملگر	توضیح	مثال	خروجی
and	هر دو شرط درست باشند	$(2 < 5) \text{ and } (3 < 4)$	True
or	یکی از شرطها درست باشد	$(6 < 5) \text{ or } (2 < 3)$	True
not	نتیجه را برعکس می‌کند	$\text{not}(5 > 2)$	False

مثلاً:

```
age = 16
grade = 9
if age > 15 and grade >= 9:
    print("دانش‌آموز دبیرستانی است")
```

4. عملگرهای انتساب (Assignment Operators)

برای دادن مقدار به متغیرها و انجام عملیات روی آنها استفاده می‌شوند.

عملگر	مثال	معنی
=	$x = 5$	مقداردهی ساده
+=	$x += 3$	$x = x + 3$
-=	$x -= 2$	$x = x - 2$
*=	$x *= 4$	$x = x * 4$
/=	$x /= 2$	$x = x / 2$
//	$x //= 2$	$x = x // 2$
%=	$x \% = 3$	$x = x \% 3$
=	$x ** = 2$	$x = x ** 2$

مثلاً:

```
x = 10
x += 5
print(x) # 15
```

5. عملگرهای عضویت (Membership Operators)

برای بررسی وجود یک مقدار در لیست، رشته یا مجموعه استفاده می‌شود.

عملگر	توضیح	مثال	خروجی
in	بررسی حضور	True a in "salam"	
not in	بررسی نبودن	True b not in "salam"	

مثلاً:

```
letters = ["a", "b", "c"]  
print("a" in letters) # True  
print("d" not in letters) # True
```

6. عملگرهای بیتی (Bitwise Operators)

برای کار روی سطح بیت (اعداد دودویی) استفاده می‌شوند.
اگرچه در برنامه‌های روزمره کمتر استفاده می‌شوند، ولی در پردازش سطح پایین کاربرد دارند.

عملگر	توضیح	مثال
&	AND بیتی	a & b
	OR بیتی	
^	XOR بیتی	a ^ b
~	NOT بیتی	a~
>>	شیفت به چپ	a << 1
<<	شیفت به راست	a >> 1

مثلاً به کد زیر توجه کنید:

```
a = 5 # (0101)
b = 3 # (0011)
print(a & b) # 1
print(a | b) # 7
```

7. تقدم عملگړها (Operator Precedence)

پايتون مثل رياضي، ترتيب اجراي عملگړها را رعايت مي‌کند.
به عنوان مثال، ضرب و تقسيم قبل از جمع و تفريق انجام مي‌شوند.
ترتيب کلي به صورت زير است:

پرانتزها ()

توان **

ضرب، تقسيم، باقيمانده، تقسيم صحيح

جمع و تفريق

مقايسه‌ها

عملگړهاي منطقي not, and, or

مثلاً:

```
result = 2 + 3 * 4
```

خروجي 14 چون اول ضرب انجام مي‌شود # print(result)

اما:

```
result = (2 + 3) * 4
```

```
print(result) # خروجی 20 چون پرانتز اولویت دارد
```

■ جمع‌بندی نهایی:

- ♦ عملگرها ابزارهای انجام محاسبات و تصمیم‌گیری در پایتون‌اند.
- ♦ عملگرهای حسابی برای ریاضیات، مقایسه‌ای برای شرطها، منطقی برای ترکیب آنها استفاده می‌شوند.
- ♦ با تسلط بر عملگرها می‌توانی برنامه‌های پیچیده‌تر و هوشمندتری بنویسی.
- ♦ تمرین زیاد با مثال‌های مختلف بهترین روش یادگیری عملگرها است.

درس هفتم

درس هفتم

ورودی و خروجی داده‌ها

در برنامه‌نویسی با پایتون، ورودی (Input) و خروجی (Output) دو مفهوم بسیار مهم هستند. ورودی یعنی دریافت اطلاعات از کاربر یا منبع دیگر، و خروجی یعنی نمایش نتیجه یا پیام به کاربر.

■ ا. خروجی (Output)

در پایتون برای نمایش متن یا نتیجه‌ی محاسبه از تابع `print()` استفاده می‌کنیم. تابع `print()` می‌تواند عدد، متن، یا چند مقدار مختلف را با هم نمایش دهد.

مثلاً:

```
print("سلام دنیا")  
print("نتیجه جمع:", 3 + 5)
```

در خط دوم، پایتون ابتدا جمع را حساب می‌کند و سپس نمایش می‌دهد.

اگر بخواهی چند چیز را با هم چاپ کنی، بین آن‌ها از `,` استفاده کن:

```
name = "علی"  
age = 14  
print("نام:", name, ", سن:", age)
```

خروجی:

نام: علی , سن: 14

می‌توانی خروجی را زیباتر کنی با استفاده از قالب‌بندی (Formatting):

```
print(f"سال دارم {age} است و {name} نام من")
```

در اینجا `f` باعث می‌شود بتوانی متغیرها را داخل رشته قرار دهی.

نکته: `f` همیشه قبل از کوتیشن می‌آید. `f" <==> "`

۲. ورودی (Input) 

برای دریافت داده از کاربر از تابع `input()` استفاده می‌کنیم.

این تابع برنامه را متوقف می‌کند تا کاربر چیزی تایپ کند.

مثلاً:

```
name = input("نام شما چیست؟")  
print("سلام", name, "خوش آمدی")
```

در این مثال، متن داخل پرانتز پیام راهنما برای کاربر است.

نکته: اینپوت برای گرفتن ورودی از کاربر می‌باشد.

نکته مهم: هر ورودی که از `input()` گرفته می‌شود، به صورت رشته (`string`) ذخیره می‌شود.

اگر بخواهی عدد دریافت کنی، باید آن را تبدیل به عدد کنی:

```
age = int(input("سن خود را وارد کنید: "))  
print("ساله می‌شوید", age + 1, "شما سال آینده")
```

در اینجا `int()` رشته را به عدد صحیح تبدیل کرده است.

اگر بخواهی عدد اعشاری بگیری، از `float()` استفاده کن:

```
height = float(input("قد خود را به متر وارد کنید"))  
print("متر است", height, "قد شما")
```

۳. ترکیب ورودی و خروجی

می‌توانی ورودی‌های مختلف را گرفته و با آن‌ها محاسبات انجام دهی:

```
a = int(input("عدد اول را وارد کنید"))  
b = int(input("عدد دوم را وارد کنید"))  
print("جمع دو عدد =", a + b)
```

۴. چند نکته کاربردی

همیشه برای خوانایی بهتر، پیامی مناسب در `input()` بنویس تا کاربر بداند چه باید وارد کند.

می‌توانی خروجی را در چند خط چاپ کنی:

```
print("سلام")  
print("به برنامه خوش آمدی")
```

یا چند خط را در یک `print` با `\n` جدا کنی:

```
print("به برنامه خوش آمدی\nسلام")
```

نکته کاربردی:

انواع داده `1 <==` عددی `2` رشته `3` بولی
* تا الان با این سه نوع داده آشنا شدیم، در آینده با انواع داده بیشتری آشنا خواهیم شد.

جمع‌بندی

♦ `print()` برای نمایش داده‌هاست (خروجی).

- ♦ input() برای دریافت داده‌هاست (ورودی).
- ♦ ورودی‌ها همیشه رشته‌اند و برای عدد باید تبدیل نوع انجام دهی.
- ♦ ترکیب ورودی و خروجی به برنامه پویایی و تعامل با کاربر می‌دهد.

درس هشتم

درس هشتم

دستورات شرطی (if, else, elif)

در پایتون، دستورات شرطی (Conditional Statements) برای تصمیم‌گیری در برنامه استفاده می‌شوند. به زبان ساده، با استفاده از شرط‌ها می‌توانیم بگوییم: «اگر فلان حالت اتفاق افتاد، فلان کار را انجام بده؛ وگرنه کار دیگری را انجام بده.»

■ ۱. ساختار اصلی شرط if

کلمه‌ی کلیدی if یعنی «اگر».

بعد از آن شرط را می‌نویسیم، و اگر آن شرط درست (True) باشد، دستورهای داخل آن اجرا می‌شوند.

مثلاً:

```
age = 18
if age >= 18:
    print("شما بزرگسال هستید")
```

در این مثال چون مقدار age برابر ۱۸ است، شرط برقرار می‌شود و پیام چاپ می‌گردد.

نکته:

در پایتون بعد از شرط، باید از علامت دونقطه (:) استفاده کنی و سپس دستورها را با تو رفتگی (indent) بنویسی.

اگر تو رفتگی رعایت نشود، خطا می‌دهد.

■ ۲. استفاده از else

گاهی می‌خواهیم اگر شرط برقرار نبود، کار دیگری انجام شود.

در این حالت از else استفاده می‌کنیم

به کد زیر توجه کنید:

```
age = 15
if age >= 18:
    print("ورود مجاز است.")
else:
    print("سن شما کافی نیست")
```

در اینجا چون شرط برقرار نیست، قسمت else اجرا می‌شود.

۳. استفاده از elif (مخفف else if) 

وقتی بیش از یک حالت مختلف داریم، از elif استفاده می‌کنیم.
یعنی اگر شرط اول برقرار نبود، شرط بعدی را بررسی کن.

مثلاً:

```
score = 85

if score >= 90:
    print("عالی")
elif score >= 75:
    print("خوب")
elif score >= 50:
    print("قبول شدید")
else:
    print("مردود شدید")
```

 توضیح کد:

در اینجا (score) به معنای نمره است.
در این مثال، فقط یکی از شرطها اجرا می‌شود (اولین شرط درست).

۴. شرط‌های ترکیبی

می‌توان چند شرط را با and, or, not ترکیب کرد:

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("ورود مجاز است.")
else:
    print("اورد غیرمجاز")
```

اینجا شرط فقط وقتی درست است که هر دو قسمت ($has_id == True$ و $age \geq 18$) برقرار باشند.

۵. شرط‌های تو در تو (Nested if)

می‌توان شرط‌ها را داخل هم نوشت.
مثلاً:

```
age = 16
if age >= 13:
    if age < 18:
        print("شما نوجوان هستید.")
    else:
        print("شما بزرگسال هستید.")
else:
    print("شما کودک هستید.")
```

در اینجا پایتون شرط‌ها را از بالا به پایین بررسی می‌کند.

۶. استفاده از شرطهای کوتاه (تک خطی) 

اگر شرط کوتاه باشد، می توان آن را در یک خط نوشت:

```
print("بزرگسال") if age >= 18 else print("نوجوان")
```

این روش فقط برای کدهای ساده کاربرد دارد.

۷. مقایسه چند مقدار در یک شرط 

می توان چند مقدار را با in بررسی کرد:

```
color = "قرمز"
if color in ["قرمز", "آبی", "سبز"]:
    print("رنگ مجاز است")
else:
    print("رنگ نامعتبر است")
```

۸. مثال واقعی 

برنامه ای برای تشخیص قبولی یا مردودی دانش آموز:

```
grade = float(input("نمره خود را وارد کنید: "))

if grade >= 90:
    print("🌟!نمره عالی")
elif grade >= 75:
    print("👍 خیلی خوب")
elif grade >= 50:
```

```
print("قبول شدید ✓")
else:
    print("مردود شدید ✗")
```

۹. نکات مهم:

در پایتون فاصله و تو رفتگی (indentation) بسیار مهم است.

فقط یکی از شاخه‌های if / elif / else اجرا می‌شود.

می‌توان چند شرط را ترکیب کرد تا منطق‌های پیچیده‌تر ساخت.

شرط‌ها فقط زمانی اجرا می‌شوند که مقدارشان True باشد.

یک سری از کلمات کاربردی درس:

If - elif - else - in - not - True - float - input

جمع‌بندی:

- ♦ دستور if برای بررسی یک شرط استفاده می‌شود.
- ♦ دستور elif برای بررسی حالت‌های اضافی.
- ♦ دستور else برای حالتی است که هیچ‌کدام برقرار نیست.
- ♦ ترکیب شرط‌ها به ما امکان تصمیم‌گیری هوشمند در برنامه را می‌دهد.

درس نهم

درس نهم

حلقه‌ها (while و for)

حلقه **for** در پایتون با اکثر زبان‌های دیگر متفاوت است. این حلقه بیشتر یک **حلقه پیمایشی (Iterative Loop)** است تا یک حلقه شمارشی سنتی. هدف اصلی آن، **تکرار روی عناصر یک توالی (Sequence)** یا یک شیء قابل پیمایش (Iterable) است.

ساختار و کاربرد

- حلقه **for** برای پیمایش بر روی عناصری مانند **لیست‌ها (Lists)**، **تاپل‌ها (Tuples)**، **رشته‌ها (Strings)**، **دیکشنری‌ها (Dictionaries)** و **مجموعه‌ها (Sets)** استفاده می‌شود.

ساختار کلی:

Python

for item in iterable:

دستوراتی که برای هر عنصر اجرا می‌شوند #

print(item)

- **نحوه عملکرد:** در هر تکرار، متغیر **item** مقدار عنصر بعدی را از **iterable** (شیء قابل پیمایش) دریافت می‌کند. این کار تا زمانی ادامه می‌یابد که تمام عناصر پیمایش شوند.

استفاده با تابع range ()

برای شبیه‌سازی حلقه‌های شمارشی سنتی که بر اساس عدد تکرار می‌شوند، از تابع داخلی **range ()** استفاده می‌شود.

شمارش ساده: range () (اعداد 0 تا 4):

Python

for i in range(5):

خروجی: 0, 1, 2, 3, 4 # print(i)

•

شروع، پایان، گام: range (start, stop, step)
Python

for j in range(2, 10, 2):

خروجی: 2, 4, 6, 8 # print(j)

- نکته: عدد **stop** (پایان) شامل حلقه نمی‌شود.

حلقه **while** در پایتون (Conditional Loop) ⌚

حلقه **while** در پایتون، دقیقاً مانند سایر زبان‌ها، یک **حلقه مبتنی بر شرط** است. بلوک کد داخل آن تا زمانی که شرط تعیین شده **درست (True)** باشد، تکرار می‌شود.

ساختار و کاربرد

- حلقه **while** زمانی استفاده می‌شود که **تعداد دفعات تکرار از قبل مشخص نیست** و تکرار منوط به برقراری یک وضعیت یا شرط منطقی است.

ساختار کلی:

Python

مقداردهی اولیه قبل از حلقه # counter = 0

while counter < 5:

دستوراتی که قرار است تکرار شوند #

print("تکرار")

(به‌روزرسانی متغیر کنترل (اجباری # counter += 1

- نحوه عملکرد: ابتدا شرط بررسی می‌شود. اگر درست باشد، بدنه حلقه اجرا شده و پس از آن، دوباره شرط بررسی می‌شود. این روند تا زمانی ادامه می‌یابد که شرط نادرست شود.
- حلقه بی‌نهایت: اگر در بدنه حلقه، متغیر **counter** به‌روزرسانی نشود یا شرط همیشه **True** بماند (مانند **while True**), یک **حلقه بی‌نهایت** (Infinite Loop) ایجاد می‌شود.

دستورات کنترلی رایج 🛑

این دستورات در هر دو حلقه **for** و **while** قابل استفاده هستند:

- **break**: **کاربرد:** اجرای **کل حلقه** را به‌طور کامل متوقف کرده و بلافاصله جریان برنامه را به اولین دستور پس از حلقه منتقل می‌کند.

مثال:

Python

```
for number in [1, 2, 3, 4, 5]:
```

```
    if number == 4:
```

```
        break
```

```
print(number) # 3, 2, 1: خروجی:
```

○

• **continue:**

○ **کاربرد:** اجرای دور فعلی حلقه را متوقف کرده و به تکرار بعدی می‌پرد. دستورات باقی‌مانده در آن دور از حلقه اجرا نخواهند شد.

مثال:

Python

```
for number in [1, 2, 3, 4, 5]:
```

```
    if number == 3:
```

```
        continue
```

```
print(number) # 5, 4, 2, 1: خروجی:
```

○

🤝 استفاده از **else** با حلقه‌ها

پایتون اجازه می‌دهد تا یک بلوک **else** به انتهای حلقه‌های **for** و **while** اضافه شود.

- **در حلقه **for**:** بلوک **else** زمانی اجرا می‌شود که حلقه به‌طور کامل و بدون استفاده از **break** تمام شود.
- **در حلقه **while**:** بلوک **else** زمانی اجرا می‌شود که شرط حلقه **False** شود.

Python

```
# مثال else در حلقه for
```

```
for i in range(3):
```

```
    print(i)
```

```
else:
```

```
print("حلقه کامل شد." # این اجرا می‌شود")
```

Python

شده است break که for در حلقه else مثال

```
for i in range(3):
```

```
    if i == 1:
```

```
        break
```

```
    print(i)
```

```
else:
```

```
    print("حلقه کامل شد." # این اجرا نمی‌شود")
```


درس دهم

لیست‌ها (Lists)

لیست‌ها (Lists) در پایتون، یکی از ساختارهای داده‌ای توالی (Sequence Data Types) و پرکاربردترین نوع مجموعه هستند. یک لیست، مجموعه‌ای مرتب و قابل تغییر از عناصر است.

تعریف و ویژگی‌های کلیدی لیست‌ها

لیست‌ها نقش اساسی در سازماندهی و مدیریت داده‌ها در پایتون دارند.

تعریف: لیست‌ها با قرار دادن عناصر در داخل کروشه `[]` و جدا کردن آن‌ها با کاما ساخته می‌شوند.

Python

```
my_list = [10, "hello", 3.14, True]
```

1. **مرتب (Ordered):** عناصر لیست، ترتیب ورود خود را حفظ می‌کنند. هر عنصری دارای یک **ایندکس (Index)** مشخص است که از **صفر (0)** شروع می‌شود.

2. **قابل تغییر (Mutable):** برخلاف رشته‌ها (Strings) یا تاپل‌ها (Tuples)، می‌توان عناصر لیست را پس از ایجاد، تغییر داد، اضافه یا حذف کرد.

3. **شامل انواع داده‌ای مختلف:** یک لیست می‌تواند شامل انواع داده‌ای متفاوت، از جمله اعداد، رشته‌ها، اشیاء و حتی لیست‌های دیگر (لیست‌های تودرتو) باشد.

4. **ایندکس‌گذاری (Indexing):** دسترسی به عناصر از طریق ایندکس انجام می‌شود.

- ایندکس مثبت: از چپ به راست و از 0 شروع می‌شود. (مثلاً `my_list[0]` برای اولین عنصر).
- ایندکس منفی: از راست به چپ و از -1 شروع می‌شود. (مثلاً `my_list[-1]` برای آخرین عنصر).

بخشی از لیست را می‌توان با استفاده از علامت دونقطه (`:`) و تعیین شروع، پایان و گام (**Slicing**) برش داد.

```
Python
numbers = [1, 2, 3, 4, 5]
print(numbers[1:4]) # خروجی: 4, 3, 2
```

متدهای اصلی برای دستکاری لیست‌ها (Modifying Lists)

لیست‌ها از توابع و متدهای قدرتمندی برای مدیریت عناصر خود پشتیبانی می‌کنند:

1. اضافه کردن عناصر:

`append(item)`: یک عنصر را در **انتهای** لیست اضافه می‌کند.

```
Python
my_list.append(6) # [10, "hello", ..., 6]
```

○

`insert(index, item)`: یک عنصر را در **ایندکس مشخص** اضافه می‌کند.

```
Python
my_list.insert(1, "new") # [10, "new", "hello", ...]
```

○

`extend(iterable)`: عناصر یک توالی دیگر (مانند لیست یا تاپل) را به **انتهای** لیست فعلی اضافه می‌کند.

```
Python
my_list.extend([7, 8]) # [..., 6, 7, 8]
```

○

2. حذف کردن عناصر:

`remove(item)`: **اولین وقوع** یک عنصر با **مقدار مشخص** را حذف می‌کند. اگر عنصر پیدا نشود، خطا (ValueError) می‌دهد.

```
Python
my_list.remove("hello")
```

○

pop(index): عنصر موجود در **ایندکس مشخص** را حذف کرده و مقدار آن را برمی‌گرداند. اگر ایندکس مشخص نشود، آخرین عنصر را حذف می‌کند.

Python

```
last_item = my_list.pop()
```

○

del Statement: برای حذف یک عنصر در ایندکس مشخص یا حذف یک برش (Slice) از لیست، یا حذف کل لیست.

Python

```
del my_list[0]
```

```
del my_list[1:3]
```

```
# del my_list # برای حذف کل متغیر لیست
```

○

clear(): تمام عناصر لیست را حذف کرده، اما خود لیست باقی می‌ماند (به لیست خالی تبدیل می‌شود).

Python

```
my_list.clear() # []
```

○

3. به‌روزرسانی عناصر:

از طریق ایندکس می‌توان مقدار یک عنصر را مستقیماً تغییر داد.

Python

```
my_list[0] = 100 # تغییر اولین عنصر
```

○

توابع و متدهای کمکی 🔍

1. **len(list):** تعداد عناصر (طول) لیست را برمی‌گرداند.

2. `count(item)`: تعداد دفعات تکرار یک عنصر مشخص در لیست را برمی‌گرداند.

```
Python  
list.count(10)
```

3. `index(item, start, end)`: ایندکس اولین وقوع یک عنصر مشخص را برمی‌گرداند. می‌توان بازه جستجو را نیز مشخص کرد.

```
Python  
list.index("hello")
```

4. `sort()`: عناصر لیست را به صورت درجا (In-place) مرتب می‌کند (لیست اصلی تغییر می‌کند). پیش‌فرض مرتب‌سازی صعودی است.

```
Python  
numbers.sort(reverse=True) # مرتب‌سازی نزولی
```

5. `sorted(list)`: یک لیست مرتب‌شده جدید برمی‌گرداند، در حالی که لیست اصلی تغییر نمی‌کند.

```
Python  
new_sorted = sorted(numbers)
```

6. `reverse()`: ترتیب عناصر لیست را به صورت درجا معکوس می‌کند (لیست اصلی تغییر می‌کند).

```
Python  
my_list.reverse()
```

7.

درک عمیق‌تر

1. عملیات روی لیست‌ها:

- الحاق (**Concatenation**): با استفاده از عملگر `+` می‌توان دو لیست را به هم چسباند. این عملیات یک لیست جدید ایجاد می‌کند.
- تکرار (**Repetition**): با استفاده از عملگر `*` می‌توان یک لیست را چند بار تکرار کرد. این عملیات یک لیست جدید ایجاد می‌کند.

2. کپی کردن لیست‌ها: از آنجایی که لیست‌ها اشیای قابل تغییر هستند، استفاده از عملگر تخصیص `=`، فقط یک ارجاع ایجاد می‌کند (Shallow Copy). برای ایجاد یک کپی واقعی (Deep Copy) باید از روش‌های زیر استفاده کرد:

- استفاده از متد `copy()`: `new_list = old_list.copy`
- استفاده از برش کامل: `new_list = old_list[:]`

3. **پیمایش با حلقه:** حلقه‌های `for` پرکاربردترین روش برای پیمایش روی عناصر لیست هستند:

Python

```
for item in my_list:
```

```
    print(item)
```

یا اگر نیاز به ایندکس عنصر هم باشد، از تابع `enumerate()` استفاده می‌شود:

Python

```
for index, value in enumerate(my_list):
```

```
    print(f"Index {index}: Value {value}")
```

درک مطلب لیست (List Comprehension) ✨

یکی از قدرتمندترین و پایتونیک‌ترین روش‌ها برای ساخت سریع لیست‌ها بر اساس یک توالی دیگر، **درک مطلب لیست** است. این روش اغلب مختصر تر و کارآمدتر از استفاده از حلقه‌های `for` سنتی است.

ساختار:

Python

```
new_list = [expression for item in iterable if condition]
```

•

مثال: ساخت لیستی از مربع اعداد زوج در یک بازه:

Python

```
squares_of_evens = [x**2 for x in range(10) if x % 2 == 0]
```

• # خروجی: `[0, 4, 16, 36, 64]`

درس یازدهم

درس یازدهم

تاپل‌ها (Tuple)

تاپل‌ها یکی از انواع داده‌های مرتب‌شده و تغییرناپذیر (Immutable) در پایتون هستند. یادگیری تاپل‌ها برای مدیریت داده‌ها و برنامه‌نویسی حرفه‌ای ضروری است.

۱. تعریف تاپل

• تاپل (Tuple) مجموعه‌ای از عناصر است که:

- مرتب شده (Ordered) است
- تغییرناپذیر (Immutable) است

```
my_tuple = (1, 2, 3)
print(my_tuple)
```

۲. تفاوت تاپل و لیست

ویژگی	تاپل	لیست
تغییرناپذیری	بله	خیر
علامت نگهداری	()	[]
سرعت	سریع‌تر	کندتر
استفاده	داده ثابت	داده قابل تغییر

۳. ایجاد تاپل

- با پرانتز:

```
tuple1 = (10, 20, 30)
tuple2 = ("a", "b", "c")
tuple3 = (1, "hello", 3.14)
```

- (بدون پرانتز) توصیه شده برای ساده خوانی:

```
tuple4 = 1, 2, 3, 4
```

۴. تاپل خالی و تک عنصری

- تاپل خالی:

```
empty_tuple = ()
```

- تاپل تک عنصری:

```
single_tuple = (5,)
print(type(single_tuple)) # <class 'tuple'>
```

دقت کنید که بدون **ویرگول**، Python آن را به عنوان عدد معمولی در نظر می‌گیرد.

۵. دسترسی به عناصر تاپل

- استفاده از ایندکس:

```
my_tuple = (10, 20, 30)
```

```
print(my_tuple[0]) # 10
print(my_tuple[1]) # 20
```

- ایندکس منفی:

```
print(my_tuple[-1]) # 30
print(my_tuple[-2]) # 20
```

۶. برش (Slicing) تاپل

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple[1:4]) # (2, 3, 4)
print(my_tuple[:3]) # (1, 2, 3)
print(my_tuple[2:]) # (3, 4, 5)
```

- نیز وجود دارد **step** امکان استفاده از:

```
print(my_tuple[::2]) # (1, 3, 5)
```

۷. تکرار و ترکیب تاپل‌ها

- **+** ترکیب تاپل‌ها با عملگر

```
tuple1 = (1, 2)
tuple2 = (3, 4)
combined = tuple1 + tuple2
print(combined) # (1, 2, 3, 4)
```

- ***: تکرار تاپل با**

```
tuple3 = (0,) * 5  
print(tuple3)  # (0, 0, 0, 0, 0)
```

۸. بررسی عضویت در تاپل

```
my_tuple = (10, 20, 30)  
print(20 in my_tuple)  # True  
print(50 not in my_tuple)  # True
```

۹. طول، بیشینه و کمینه

```
numbers = (5, 10, 15, 20)  
print(len(numbers))  # 4  
print(max(numbers))  # 20  
print(min(numbers))  # 5
```

۱۰. حلقه‌ها و تاپل

- **for loop** برای دسترسی به عناصر

```
fruits = ("apple", "banana", "cherry")  
for fruit in fruits:  
    print(fruit)
```

- برای گرفتن ایندکس با **enumerate**:

```
for index, fruit in enumerate(fruits):  
    print(index, fruit)
```

۱۱. جستجو در تاپل

- **index** استفاده از:

```
fruits = ("apple", "banana", "cherry")  
print(fruits.index("banana")) # 1
```

- برای تعداد تکرار عنصر **count** استفاده از:

```
numbers = (1, 2, 2, 3)  
print(numbers.count(2)) # 2
```

۱۲. توابع داخلی کاربردی برای تاپل

- **len()**: تعداد عناصر
- **sum()**: جمع اعداد (اگر عددی باشند)
- **max(), min()**: بیشینه و کمینه
- **sorted()**: مرتب‌سازی (خروجی لیست می‌دهد)

```
nums = (4, 1, 7)  
print(sorted(nums)) # [1, 4, 7]
```

۱۳. تغییرناپذیری تاپل

- تاپل غیر قابل تغییر است:

```
my_tuple = (1, 2, 3)
# my_tuple[0] = 100 # خطا می‌دهد
```

- اما اگر تاپل شامل لیست یا آبجکت قابل تغییر باشد، می‌توان محتویات آن را تغییر داد:

```
my_tuple = ([1, 2], 3)
my_tuple[0][0] = 10
print(my_tuple) # ([10, 2], 3)
```

۱۴. unpacking یا باز کردن تاپل

```
my_tuple = (1, 2, 3)
a, b, c = my_tuple
print(a, b, c) # 1 2 3
```

- کردن عناصر ignore برای:

```
x, _, y = (10, 20, 30)
print(x, y) # 10 30
```

۱۵. توپل‌های تو در تو

```
nested_tuple = (1, (2, 3), 4)
```

```
print(nested_tuple[1])      # (2, 3)
print(nested_tuple[1][0])   # 2
```

۱۶. تبدیل لیست به تاپل و بالعکس

```
my_list = [1, 2, 3]
my_tuple = tuple(my_list)
print(my_tuple)  # (1, 2, 3)

new_list = list(my_tuple)
print(new_list)  # [1, 2, 3]
```

۱۷. تاپل و حافظه

- تاپل‌ها به دلیل **تغییر ناپذیری** سریع‌تر و کم‌حجم‌تر از لیست‌ها هستند
 - مناسب برای داده‌هایی که **تغییر نمی‌کنند**
-

۱۸. کاربردهای رایج تاپل

1. بازگرداندن چند مقدار از یک تابع:

```
def get_min_max(nums):
    return min(nums), max(nums)

result = get_min_max([1, 5, 10])
print(result)  # (1, 10)
```

2. نگهداری داده‌های ثابت:

```
colors = ("red", "green", "blue")
```

3. کلید در دیکشنری (dict) زیرا تاپل قابل هاش شدن است:

```
my_dict = {(1, 2): "value"}  
print(my_dict[(1, 2)]) # value
```

۱۹. توابع با تاپل به عنوان ورودی و خروجی

- تاپل به راحتی می‌تواند به توابع پاس داده شود

```
def sum_tuple(t):  
    return sum(t)
```

```
numbers = (1, 2, 3)  
print(sum_tuple(numbers)) # 6
```

۲۰. نکات مهم و بهترین کاربرد تاپل

- داده‌های ثابت و غیرقابل تغییر را با تاپل ذخیره کنید
- برای بازگرداندن چند مقدار از توابع از تاپل استفاده کنید
- برای استفاده به عنوان کلید دیکشنری حتماً از تاپل استفاده کنید

درس دوازدهم

درس دوازدهم

دیکشنری‌ها (Dictionary)

دیکشنری‌ها یکی از پرکاربردترین نوع داده‌ها در پایتون هستند که برای ذخیره داده‌ها به شکل جفت کلید-مقدار (Key-Value) استفاده می‌شوند. یادگیری دیکشنری‌ها برای برنامه‌نویسی حرفه‌ای ضروری است.

۱. تعریف دیکشنری

- دیکشنری مجموعه‌ای مرتب نشده، قابل تغییر و قابل گسترش است.
- هر عنصر شامل کلید (Key) و مقدار (Value) است.

```
my_dict = {"name": "Ali", "age": 25, "city": "Tehran"}  
print(my_dict)
```

۲. ویژگی‌های کلیدی دیکشنری

- کلیدها یکتا (Unique) هستند
- کلیدها باید قابل هش باشند (مثل رشته‌ها، اعداد، تاپل‌ها)
- مقادیر می‌توانند هر نوع داده‌ای باشند
- دیکشنری قابل تغییر (Mutable) است

۳. ایجاد دیکشنری

- با آکولاد {}:

```
student = {"name": "Sara", "age": 20, "grade": "A"}
```

- با تابع `dict()`:

```
student2 = dict(name="Ali", age=25, city="Tehran")
```

- دیکشنری خالی:

```
empty_dict = {}
```

۴. دسترسی به مقادیر

- با استفاده از کلید:

```
print(student["name"]) # Sara
```

- استفاده از متد `get()`:

```
print(student.get("age")) # 20
print(student.get("phone", 0)) # 0 اگر کلید موجود نباشد
```

۵. تغییر مقادیر

```
student["age"] = 21
student["city"] = "Mashhad"
print(student)
```

۶. اضافه کردن عناصر جدید

```
student["phone"] = "09120000000"  
print(student)
```

۷. حذف عناصر

- استفاده از `del`:

```
del student["grade"]
```

- استفاده از `pop()`:

```
age = student.pop("age")  
print(age) # 21
```

- حذف آخرین عنصر (Python 3.7+) با `popitem()`:

```
last_item = student.popitem()  
print(last_item)
```

۸. بررسی عضویت

- کلیدها:

```
print("name" in student) # True  
print("grade" not in student) # True
```

- مقادیر با `values()` بررسی می‌شوند:

```
print("Sara" in student.values()) # True
```

۹. طول دیکشنری

```
print(len(student)) # تعداد کلیدها
```

۱۰. حلقه‌ها و دیکشنری

- حلقه روی کلیدها:

```
for key in student:  
    print(key)
```

- حلقه روی مقادیر:

```
for value in student.values():  
    print(value)
```

- حلقه روی کلید و مقدار:

```
for key, value in student.items():  
    print(key, value)
```

۱۱. متدهای مهم دیکشنری

- `keys()`: لیست کلیدها

```
print(student.keys())
```

- `values()`: لیست مقادیر

```
print(student.values())
```

- `items()`: لیست کلید-مقدار

```
print(student.items())
```

- `copy()`: کپی دیکشنری

```
student_copy = student.copy()
```

- `update()`: اضافه کردن یا تغییر مقادیر

```
student.update({"grade": "B", "city": "Shiraz"})
```

۱۲. توابع کمکی با دیکشنری

- `len()`: تعداد عناصر

- `del` و `pop()`: حذف عناصر

- `clear()`: پاک کردن دیکشنری

```
student.clear()
```

```
print(student) # {}
```

۱۳. دیکشنری‌های تو در تو (Nested Dictionary)

- دیکشنری‌ها می‌توانند عناصر دیکشنری دیگر باشند:

```
students = {  
    "sara": {"age": 20, "grade": "A"},  
    "ali": {"age": 25, "grade": "B"}  
}
```

```
print(students["sara"]["grade"]) # A
```

۱۴. استفاده از دیکشنری برای توابع

- بازگرداندن چند مقدار:

```
def person_info():  
    return {"name": "Ali", "age": 30, "city": "Tehran"}
```

```
info = person_info()  
print(info["name"]) # Ali
```

۱۵. کلیدهای قابل هش

- کلیدها می‌توانند عدد، رشته، تاپل یا انواع **immutable** باشند
- مثال از کلید تاپل:

```
my_dict = {(1, 2): "point"}  
print(my_dict[(1, 2)])
```

- لیست یا دیکشنری نمی‌توانند کلید باشند:

```
# { [1,2]: "value" } # خطا می‌دهد
```

۱۶. مقایسه دیکشنری‌ها

- مقایسه دو دیکشنری بر اساس **کلید-مقدار** انجام می‌شود:

```
dict1 = {"a": 1, "b": 2}  
dict2 = {"b": 2, "a": 1}  
print(dict1 == dict2) # True
```

- ترتیب کلیدها مهم نیست (+Python 3.7)
- دیکشنری‌ها mutable هستند و می‌توان آن‌ها را تغییر داد

۱۷. مرتب‌سازی دیکشنری

- مرتب‌سازی بر اساس کلید:

```
my_dict = {"b": 2, "a": 1, "c": 3}  
sorted_keys = sorted(my_dict)  
print(sorted_keys) # ['a', 'b', 'c']
```

- مرتب‌سازی بر اساس مقدار

```
sorted_values = sorted(my_dict.items(), key=lambda x: x[1])
print(sorted_values) # [('a', 1), ('b', 2), ('c', 3)]
```

۱۸. دیکشنری و حلقه‌های شرطی

```
student = {"name": "Ali", "age": 25}
```

```
if "age" in student:
    print("سن موجود است", student["age"])
else:
    print("سن موجود نیست")
```

۱۹. کاربردهای رایج دیکشنری

1. **نگهداری داده‌های کلید-مقدار:**
مانند شماره تلفن، مشخصات دانش‌آموزان
2. **بازگرداندن چند مقدار از توابع:**
به جای استفاده از تاپل یا لیست
3. **داده‌های JSON و API:**
دیکشنری‌ها مشابه JSON هستند و برای کار با API بسیار کاربرد دارند
4. **حسابداری و آمار:**
برای شمارش تکرار عناصر یا دسته‌بندی داده‌ها

۲۰. نکات مهم دیکشنری

- کلیدها یکتا و immutable هستند
- دیکشنری‌ها mutable و قابل تغییر هستند
- می‌توان از دیکشنری برای ساختاردهی داده‌های پیچیده استفاده کرد
- دیکشنری‌ها سرعت دسترسی بالایی دارند

درس سیزدهم

درس سیزدهم

مجموعه‌ها (Set)

در پایتون، **Set (مجموعه)** یکی از انواع داده‌ای مهم است که برای ذخیره‌ی مجموعه‌ای از مقادیر **منحصر به فرد** استفاده می‌شود. یعنی در یک Set هیچ عنصر تکراری وجود ندارد.

Set مانند لیست و تاپل، مجموعه‌ای از داده‌ها را در خود نگه می‌دارد، اما تفاوت اصلی آن در این است که:

- ترتیب عناصر در Set **مهم نیست** (غیرقابل مرتب‌سازی).
- مقادیر تکراری در آن **به صورت خودکار حذف می‌شوند**.

نحوه ساخت Set

برای ساخت یک مجموعه کافی است عناصر را داخل `{ }` بنویسیم:

```
numbers = {1, 2, 3, 4, 5}
print(numbers)
```

اگر مقدار تکراری اضافه کنیم، به صورت خودکار حذف می‌شود:

```
nums = {1, 2, 2, 3, 4, 4}
print(nums)    # خروجی: {1, 2, 3, 4}
```

ایجاد مجموعه خالی

برای ایجاد یک Set خالی باید از تابع `set()` استفاده کنیم، چون `{ }` به طور پیش‌فرض یک دیکشنری است:

```
empty_set = set()
print(type(empty_set))    # <class 'set'>
```

تبدیل سایر نوع‌ها به Set

می‌توان لیست، تاپل یا رشته را به Set تبدیل کرد تا عناصر تکراری حذف شوند:

```
list1 = [1, 2, 2, 3, 4, 4]
unique = set(list1)
print(unique) # {1, 2, 3, 4}

chars = set("hello")
print(chars) # {'h', 'e', 'l', 'o'}
```

افزودن و حذف عناصر

برای افزودن از `add()` و برای حذف از `remove()` یا `discard()` استفاده می‌کنیم:

```
s = {1, 2, 3}
s.add(4)
print(s) # {1, 2, 3, 4}

s.remove(2)
print(s) # {1, 3, 4}
```

```
# اگر مقدار وجود نداشته باشد:
# s.remove(10) خطا می‌دهد!
s.discard(10) # خطا نمی‌دهد اما
```

حذف تصادفی با `pop()`

تابع `pop()` یک عنصر تصادفی را از مجموعه حذف می‌کند:

```
fruits = {"apple", "banana", "cherry"}
removed = fruits.pop()
print("حذف شده:", removed)
```

```
print(fruits)
```

پاک کردن کل مجموعه

برای خالی کردن مجموعه از `clear()` استفاده می‌کنیم:

```
items = {1, 2, 3}
items.clear()
print(items)  # set()
```

عملگرهای مجموعه‌ای

مجموعه‌ها در پایتون از عملگرهای ریاضی پشتیبانی می‌کنند، مانند اجتماع، اشتراک، تفاضل و ...

اجتماع (Union)

تمام عناصر دو مجموعه را ترکیب می‌کند (بدون تکرار):

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)           # {1, 2, 3, 4, 5}
print(a.union(b))      # همان نتیجه
```

اشتراک (Intersection)

عناصری که در هر دو مجموعه وجود دارند:

```
print(a & b)           # {3}
print(a.intersection(b))
```

تفاضل (Difference)

عناصری که فقط در مجموعه اول هستند:

```
print(a - b)           # {1, 2}
```

```
print(a.difference(b))
```

تفاضل متقارن (Symmetric Difference)

عناصری که فقط در یکی از مجموعه‌ها وجود دارند:

```
print(a ^ b)          # {1, 2, 4, 5}
print(a.symmetric_difference(b))
```

بررسی عضویت

برای بررسی وجود یک عنصر از `in` استفاده می‌کنیم:

```
nums = {10, 20, 30}
print(10 in nums)    # True
print(40 in nums)    # False
```

مقایسه مجموعه‌ها

می‌توان بررسی کرد آیا یک مجموعه زیرمجموعه‌ی مجموعه‌ی دیگر است یا خیر:

```
a = {1, 2}
b = {1, 2, 3, 4}

print(a.issubset(b))    # True
print(b.issuperset(a))  # True
print(a.isdisjoint({5})) # True (هیچ اشتراکی ندارند)
```

کاربردهای مفید Set

۱. حذف داده‌های تکراری از یک لیست:

```
data = [1, 2, 2, 3, 4, 4, 5]
unique_data = list(set(data))
print(unique_data)
```

۲. بررسی حروف یکتا در یک رشته:

```
word = "programming"
if len(set(word)) == len(word):
    print("تمام حروف یکتا هستند")
else:
    print("حروف تکراری وجود دارد")
```

۳. فیلتر کردن داده‌ها:

```
names = ["Ali", "Sara", "Ali", "Reza"]
unique_names = set(names)
for name in unique_names:
    print(name)
```

نکات پایانی

عناصر Set باید قابل هش (**hashable**) باشند؛ یعنی نمی‌توان لیست یا دیکشنری داخل Set گذاشت.

```
s = {1, (2, 3)} # مجاز است
# s = {1, [2, 3]} خطا می‌دهد ←
```

- چون Set ترتیب ندارد، نمی‌توان با ایندکس به آن دسترسی داشت.

جمع‌بندی

- ✓ **Set** در پایتون ابزاری عالی برای ذخیره‌ی داده‌های بدون تکرار است.
- ✓ از عملیات ریاضی مجموعه‌ها پشتیبانی می‌کند.
- ✓ سرعت بالایی در جست‌وجو و بررسی عضویت دارد.
- ✓ اما ترتیب و ایندکس در آن وجود ندارد.

درس چهاردهم

درس چهاردهم

تعریف و فراخوانی توابع

تابع (Function) یکی از مهم‌ترین مفاهیم در برنامه‌نویسی است. تابع مجموعه‌ای از دستورات است که برای انجام یک کار خاص نوشته می‌شود. به جای تکرار چند خط کد در بخش‌های مختلف برنامه، می‌توان آن‌ها را در قالب یک تابع نوشت و هر زمان لازم بود، فقط نام آن تابع را فراخوانی کرد.

در پایتون، تعریف تابع با کلمه‌ی کلیدی **def** آغاز می‌شود. ساختار کلی یک تابع به صورت زیر است:

```
def name(parameters):  
    # بدنه تابع  
    دستورات  
    return مقدار_بازگشتی
```

- ♦ **name** نام تابع است.
- ♦ **parameters** ورودی‌هایی هستند که تابع دریافت می‌کند.
- ♦ دستور **return** مقداری را به محل فراخوانی برمی‌گرداند (اختیاری است).

مثلاً تابعی ساده برای جمع دو عدد:

```
def sum_numbers(a, b):  
    result = a + b  
    return result
```

برای **فراخوانی** تابع، فقط کافی است نام آن را بنویسیم و ورودی‌های لازم را ارسال کنیم:

```
answer = sum_numbers(5, 7)  
print(answer)
```

خروجی برنامه عدد ۱۲ خواهد بود.

♦ توابع بدون ورودی

گاهی تابع نیاز به ورودی ندارد و فقط عملی را انجام می‌دهد.

```
def say_hello():  
    print("سلام! خوش آمدی")
```

فراخوانی:

```
say_hello()
```

♦ توابع بدون مقدار بازگشتی

تابع می‌تواند کاری انجام دهد ولی چیزی برنگرداند:

```
def print_sum(a, b):  
    print("جمع:", a + b)
```

وقتی تابع `print sum(3, 4)` فراخوانی شود، فقط نتیجه را چاپ می‌کند ولی چیزی برنمی‌گرداند.

♦ پارامترهای پیش‌فرض (Default Parameters)

در پایتون می‌توان برای پارامترها مقدار پیش‌فرض تعیین کرد:

```
def greet(name="دوست من"):  
    print("سلام", name)
```

اکنون هر دو فراخوانی زیر معتبرند:

```
greet()  
greet("امیرعلی")
```

♦ توابع با تعداد نامشخص آرگومان

اگر ندانیم چند ورودی ممکن است ارسال شود، می‌توانیم از `args*` استفاده کنیم:

```
def total_sum(*numbers):  
    return sum(numbers)
```

اکنون می‌توان تابع را با هر تعداد عدد فراخوانی کرد:

```
print(total_sum(1, 2, 3))  
print(total_sum(4, 5, 6, 7, 8))
```

♦ استفاده از `kwargs**`

اگر ورودی‌ها به صورت کلید و مقدار باشند، از `kwargs**` استفاده می‌کنیم:

```
def print_info(**person):  
    for key, value in person.items():  
        print(key, ":", value)
```

فراخوانی:

```
print_info(name="Fatemeh", age=10, city="Tehran")
```

♦ بازگشت چند مقدار از تابع

در پایتون می‌توان چند مقدار را هم‌زمان بازگرداند:

```
def calculate(a, b):  
    return a + b, a - b, a * b
```

و هنگام فراخوانی:

```
sum_, diff, mult = calculate(10, 5)  
print(sum_, diff, mult)
```

♦ محدوده متغیرها (Scope)

متغیرهایی که درون تابع تعریف می‌شوند، فقط در همان تابع قابل دسترسی هستند.

```
def test():  
    x = 10  
    print(x)
```

```
test()
```

```
# print(x) ← در خارج از تابع شناخته شده نیست x خطا! چون
```

♦ استفاده از متغیرهای سراسری

اگر بخواهیم درون تابع به متغیر بیرونی دسترسی داشته باشیم، باید از کلیدواژه‌ی `global` استفاده کنیم:

```
x = 5
```

```
def change():  
    global x  
    x = 20
```

```
change()  
print(x)
```

♦ توابع بازگشتی (Recursive Functions)

تابعی که خودش را فراخوانی کند، بازگشتی نام دارد.

مثلاً تابع محاسبه فاکتوریل:

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

فراخوانی:

```
print(factorial(5))
```

♦ توابع لامبدا (Lambda Functions)

توابع کوتاه و بی‌نام هستند که با `lambda` تعریف می‌شوند:

```
square = lambda x: x ** 2  
print(square(5))
```

♦ اهمیت توابع

توابع باعث می‌شوند:

1. کدها تکراری نباشند.
2. خوانایی و نظم برنامه بالا برود.
3. نگهداری و تغییر کد راحت‌تر شود.
4. بتوان برنامه را به بخش‌های کوچکتر و قابل‌مدیریت تقسیم کرد.

در نهایت، یادگیری کامل توابع گامی اساسی برای فهم مفاهیم شی گرای، ماژول‌ها و الگوریتم‌های پیچیده‌تر است.

تقریباً هیچ برنامه‌ی واقعی بدون توابع نوشته نمی‌شود!

درس پانزدهم

درس پانزدهم

پارامترها و مقدار بازگشتی

تابع در برنامه‌نویسی مانند یک ماشین پردازشگر است. داده‌هایی را از بیرون دریافت می‌کند، روی آن‌ها عملی انجام می‌دهد، و در نهایت نتیجه‌ای را برمی‌گرداند.

۱. مفهوم پارامتر

پارامتر (Parameter) در واقع ورودی تابع است. وقتی تابعی تعریف می‌کنیم، معمولاً می‌خواهیم اطلاعاتی از بیرون دریافت کند تا بر اساس آن‌ها کاری انجام دهد.

ساختار کلی تعریف تابع با پارامترها:

```
def function_name(parameter1, parameter2, ...):  
    # بدنه تابع  
    ...
```

وقتی تابع فراخوانی می‌شود، مقادیر واقعی به این پارامترها ارسال می‌شود که به آن‌ها آرگومان (Argument) می‌گویند.

۲. مثال ساده

```
def greet(name):  
    print("سلام", name)
```

در اینجا `name` یک پارامتر است. اگر تابع را فراخوانی کنیم:

```
greet("فاطمه")
```

خروجی خواهد بود:

سلام فاطمه

در این مثال، "فاطمه" یک آرگومان است که به پارامتر **name** داده شده است.

♦ ۳. چند پارامتر در یک تابع

تابع می‌تواند چند ورودی داشته باشد:

```
def introduce(name, age, city):  
    print("من", name, "هستم", age, "سالم و ساکن", city)
```

فراخوانی تابع:

```
introduce("امیرعلی", 14, "قم")
```

♦ ۴. اهمیت ترتیب پارامترها

وقتی پارامترها چند تا هستند، ترتیب آرگومان‌ها در فراخوانی باید با ترتیب تعریف تابع یکسان باشد.

```
def show(a, b, c):  
    print(a, b, c)
```

```
show(1, 2, 3)    # درست  
show(3, 2, 1)   # ترتیب متفاوت، نتیجه متفاوت!
```

♦ ۵. پارامترهای با مقدار پیش‌فرض

گاهی می‌خواهیم پارامتر اختیاری باشد.
در این حالت برای آن مقدار پیش‌فرض (**Default Value**) تعیین می‌کنیم:

```
def greet(name="دوست من"):  
    print("سلام", name)
```

اکنون می‌توان تابع را با یا بدون ورودی فراخوانی کرد:

```
greet()          # خروجی: سلام دوست من
greet("زهرا")    # خروجی: سلام زهرا
```

♦ ۶. پارامترهای متغیر (**args***)

اگر ندانیم کاربر چند ورودی می‌دهد، از **args*** استفاده می‌کنیم:

```
def show_numbers(*numbers):
    print(numbers)
```

فراخوانی:

```
show_numbers(1, 2, 3, 4)
```

در خروجی خواهیم دید:

```
(1, 2, 3, 4)
```

در واقع، **args*** تمام ورودی‌ها را به شکل **تاپل (tuple)** ذخیره می‌کند.

♦ ۷. پارامترهای کلیددار (**kwargs****)

برای دریافت ورودی‌هایی به صورت کلید و مقدار از **kwargs**** استفاده می‌کنیم:

```
def info(**data):
    print(data)
```

فراخوانی:

```
info(name="Fatemeh", age=10, city="Tehran")
```

خروجی:

```
{'name': 'Fatemeh', 'age': 10, 'city': 'Tehran'}
```

♦ ۸. تفاوت آرگومان و پارامتر

واژه	زمان استفاده	توضیح
پارامتر (Parameter)	هنگام تعریف تابع	نام متغیری که ورودی را دریافت می‌کند
آرگومان (Argument)	هنگام فراخوانی تابع	مقدار واقعی که به تابع داده می‌شود

♦ ۹. مقدار بازگشتی (return)

تابع می‌تواند نتیجه‌ی خود را **بازگرداند** تا در جای دیگری استفاده شود. این کار با دستور **return** انجام می‌شود.

مثال:

```
def sum_numbers(a, b):  
    return a + b
```

فراخوانی:

```
x = sum_numbers(5, 7)  
print(x)
```

نتیجه:

12

♦ ۱۰. تابع بدون مقدار بازگشتی

اگر در تابع `return` ننویسیم، مقدار پیش فرض بازگشتی `None` است.

```
def say_hello():  
    print("سلام!")  
  
result = say_hello()  
print(result)
```

خروجی:

!سلام

None

♦ ۱۱. بازگرداندن چند مقدار

در پایتون می توان چند مقدار را باهم برگرداند:

```
def calculate(a, b):  
    return a + b, a - b, a * b
```

فراخوانی:

```
sum_, diff, mult = calculate(4, 2)  
print(sum_, diff, mult)
```

خروجی:

6 2 8

♦ ۱۲. مقدار بازگشتی در تصمیم‌گیری

می‌توان از مقدار بازگشتی برای کنترل جریان برنامه استفاده کرد:

```
def is_even(num):  
    return num % 2 == 0  
  
if is_even(10):  
    print("عدد زوج است")  
else:  
    print("عدد فرد است")
```

♦ ۱۳. چند return در یک تابع

می‌توان در یک تابع چند حالت بازگشتی داشت:

```
def check_sign(n):  
    if n > 0:  
        return "مثبت"  
    elif n < 0:  
        return "منفی"  
    else:  
        return "صفر"
```

♦ ۱۴. return باعث خروج از تابع می‌شود

وقتی تابع به دستور return می‌رسد، اجرای آن متوقف می‌شود:

```
def test():  
    print("اول")  
    return  
    print("دوم")  # این خط اجرا نمی‌شود
```

```
test()
```

♦ ۱۵. بازگشت توابع تو در تو

تابع می‌تواند تابع دیگری را فراخوانی کرده و نتیجه آن را بازگرداند:

```
def square(x):  
    return x * x  
  
def double_square(x):  
    return 2 * square(x)
```

♦ ۱۶. مقدار بازگشتی در توابع بازگشتی (Recursive)

در توابع بازگشتی، مقدار `return` بسیار مهم است.
مثلاً در فاکتوریل:

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

♦ ۱۷. نکته مهم

اگر تابعی مقدار بازگشتی دارد، حتماً باید آن را در فراخوانی ذخیره یا استفاده کنیم؛ وگرنه محاسبه بی‌فایده می‌شود.

```
sum_numbers(2, 3)    # بی‌اثر  
print(sum_numbers(2, 3)) # مفید
```

♦ ۱۸. جمع‌بندی

- ✓ پارامترها: ورودی‌های تابع هستند.
 - ✓ مقدار بازگشتی: خروجی تابع است.
 - ✓ `return` می‌تواند یک یا چند مقدار برگرداند.
 - ✓ توابع بدون `return` مقدار `None` تولید می‌کنند.
 - ✓ پارامترها می‌توانند پیش‌فرض، متغیر یا کلیددار باشند.
-

♦ ۱۹. نتیجه‌گیری

- درک دقیق پارامترها و مقدار بازگشتی باعث می‌شود:
- برنامه‌نویسی تمیزتر و قابل‌درک‌تری داشته باشیم،
 - توابع قابل‌استفاده مجدد (Reusable) بنویسیم،
 - و بتوانیم داده‌ها را بین بخش‌های مختلف برنامه جابه‌جا کنیم.

درس شانزدهم

درس شانزدهم

کتاب خانه و ماژول

در برنامه نویسی پایتون، هرچه برنامه بزرگتر شود، نگهداری کدها در یک فایل دشوار می شود. برای مدیریت بهتر و استفاده مجدد از کدها، پایتون مفاهیم **ماژول (Module)** و **کتابخانه (Library/Package)** را ارائه کرده است.

۱. ماژول چیست؟

ماژول یک فایل پایتون (**.py**) است که شامل مجموعه ای از توابع، کلاس ها و متغیرها می باشد. هدف از ماژول، تقسیم برنامه به بخش های کوچک و قابل استفاده مجدد است.

مثال ساده از ماژول:

```
# فایل: my_module.py

def greet(name):
    print("سلام", name)

def add(a, b):
    return a + b
```

۲. فراخوانی ماژول

برای استفاده از توابع یا کلاس های ماژول در فایل دیگر، از **import** استفاده می کنیم:

```
# فایل: main.py

import my_module

my_module.greet("فاطمه")
print(my_module.add(5, 7))
```

خروجی:

سلام فاطمه

12

۳. وارد کردن توابع خاص از ماژول

می‌توان تنها یک تابع یا کلاس را وارد کرد:

```
from my_module import add
```

```
print(add(10, 20))
```

در این حالت نیاز نیست نام ماژول را قبل از تابع بنویسیم.

۴. استفاده از Alias برای ماژول

گاهی نام ماژول طولانی است یا می‌خواهیم نام کوتاه‌تری استفاده کنیم:

```
import my_module as mm
```

```
print(mm.add(3, 4))
```

۵. پارامتر و مقدار بازگشتی در ماژول‌ها

ماژول‌ها می‌توانند شامل توابع با پارامتر و مقدار بازگشتی باشند:

```
# my_module.py
```

```
def multiply(a, b):  
    return a * b
```

```
def calculate(a, b):  
    return a + b, a - b, a * b
```

۶. ماژول شامل کلاس

```
# my_module.py  
  
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def info(self):  
        print(f"نام: {self.name}, سن: {self.age}")
```

فراخوانی:

```
from my_module import Person  
  
p = Person("امیرعلی", 14)  
p.info()
```

۷. ماژول با متغیرها

می‌توان متغیرهای سراسری در ماژول تعریف کرد:

```
# my_module.py  
  
pi = 3.14159
```

```
def circle_area(r):  
    return pi * r * r
```

۸. استفاده از `__name__ == "__main__"`

گاهی می‌خواهیم ماژول هم قابلیت اجرا به صورت مستقل داشته باشد و هم وارد شدن در برنامه‌های دیگر:

```
# my_module.py
```

```
def greet(name):  
    print("سلام", name)
```

```
if __name__ == "__main__":  
    greet("دنيا")
```

۹. کتابخانه چیست؟

کتابخانه (Library / Package) مجموعه‌ای از ماژول‌ها است که در یک پوشه سازماندهی شده‌اند. پوشه کتابخانه شامل یک فایل ویژه `__init__.py` است که آن پوشه را به عنوان پکیج معرفی می‌کند.

ساختار نمونه:

```
my_library/  
    __init__.py  
    math_ops.py  
    string_ops.py
```

- `math_ops.py`: توابع ریاضی
- `string_ops.py`: توابع رشته‌ای

- معرفی پکیج: `__init__.py`

۱۰. نمونه ماژول ریاضی

```
# math_ops.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

۱۱. نمونه ماژول رشته‌ای

```
# string_ops.py

def greet(name):
    return f"سلام {name}!"

def shout(text):
    return text.upper()
```

۱۲. فراخوانی از کتابخانه

```
from my_library import math_ops, string_ops

print(math_ops.add(5, 7))
print(string_ops.greet("زهرا"))
print(string_ops.shout("سلام"))
```

۱۳. نصب و استفاده از کتابخانه‌های آماده

پایتون شامل بسیاری کتابخانه آماده است که می‌توان بدون نوشتن کد از آن‌ها استفاده کرد، مانند:

- `math` → توابع ریاضی
- `random` → تولید اعداد تصادفی
- `os` → دسترسی به سیستم‌عامل
- `datetime` → مدیریت تاریخ و زمان

مثال:

```
import math
print(math.sqrt(25)) # جذر عدد
print(math.pi)      # عدد پی
```

```
import random
print(random.randint(1, 10)) # عدد تصادفی بین ۱ تا ۱۰
```

۱۴. استفاده از توابع کتابخانه‌ها

می‌توان فقط یک تابع خاص را وارد کرد:

```
from math import sqrt, ceil

print(sqrt(49))
print(ceil(3.14)) # گرد کردن به بالا
```

۱۵. ایجاد ماژول با توابع پیشرفته

```
# my_module.py
```

```
def factorial(n):  
    """ به صورت بازگشتی n محاسبه فاکتوریل عدد """  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)
```

۱۶. سازماندهی ماژول‌ها در کتابخانه

با تقسیم کدها به چند ماژول، نگهداری برنامه آسان می‌شود و امکان استفاده مجدد از توابع وجود دارد.

```
my_package/  
    __init__.py  
    math_ops.py  
    string_ops.py  
    utils.py
```

- `utils.py` می‌تواند شامل توابع کمکی باشد.

۱۷. نمونه `utils.py`

```
def greet_all(names):  
    for name in names:  
        print(f"سلام {name}!")
```

فراخوانی:

```
from my_package import utils  
  
utils.greet_all(["امیرعلی", "فاطمه"])
```

۱۸. نکات مهم در استفاده از ماژول و کتابخانه

۱. نام ماژول و پکیج باید معتبر و بدون فاصله باشد.
۲. هر ماژول باید یک **مسئولیت مشخص** داشته باشد (یک کار خاص انجام دهد).
۳. توابع و کلاس‌ها باید **مستند (Docstring)** داشته باشند.
۴. از `__name__ == "__main__"` برای تست ماژول استفاده کنید.
۵. از **Alias** برای کوتاه کردن نام ماژول‌ها بهره ببرید.

۱۹. مثال عملی: کتابخانه مدیریت دانش‌آموز

```
student_library/  
    __init__.py  
    student_ops.py
```

student_ops.py:

```
students = []
```

```
def add_student(name, age):  
    students.append({"name": name, "age": age})
```

```
def list_students():  
    for s in students:  
        print(s["name"], "-", s["age"])
```

فراخوانی:

```
from student_library import student_ops as so
```

```
so.add_student("فاطمه", 10)
```



```
so.add_student("14", "امیرعلی")  
so.list_students()
```

۲۰. بهترین روش‌ها در طراحی کتابخانه

- توابع کوتاه، واضح و با مسئولیت مشخص باشند.
 - کلاس‌ها و توابع مرتبط در یک ماژول قرار گیرند.
 - از نام‌های قابل فهم و استاندارد استفاده کنید.
 - مستند سازی با Docstring برای هر تابع و کلاس انجام شود.
 - امکان استفاده هم به صورت مستقیم و هم به صورت وارد شده در پروژه‌های دیگر فراهم باشد.
-

۲۱. جمع‌بندی

- **ماژول** = یک فایل `py` با توابع، کلاس‌ها و متغیرها
- **کتابخانه / پکیج** = مجموعه‌ای از ماژول‌ها
- استفاده از ماژول‌ها و کتابخانه‌ها باعث **خوانایی، نظم، کاهش تکرار کد و قابلیت استفاده مجدد** می‌شود.
- پایتون شامل کتابخانه‌های آماده و محبوب زیادی است که می‌توان با `import` از آن‌ها بهره برد.
- می‌توان **ماژول شخصی و کتابخانه شخصی** ساخت و بین پروژه‌ها استفاده کرد.

درس هفدهم

درس هفدهم

ساخت ماژول شخصی

در برنامه‌نویسی پایتون، وقتی برنامه‌ها بزرگ می‌شوند، نگهداری کدها در یک فایل واحد مشکل می‌شود. برای مدیریت بهتر، کدها را می‌توان به ماژول‌ها تقسیم کرد.

ماژول (Module) مجموعه‌ای از توابع، کلاس‌ها و متغیرها است که در یک فایل `py` قرار می‌گیرد و می‌توان آن را در فایل‌های دیگر استفاده کرد.

۱. ایجاد یک ماژول ساده

فرض کنید می‌خواهیم یک ماژول با نام `my_module.py` بسازیم. در این ماژول چند تابع ساده تعریف می‌کنیم:

```
# فایل: my_module.py

def greet(name):
    print("سلام", name)

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

۲. فراخوانی ماژول

برای استفاده از ماژول در یک فایل دیگر، از دستور `import` استفاده می‌کنیم:

```
# فایل: main.py

import my_module
```

```
my_module.greet("فاطمه")
result = my_module.add(5, 7)
print("جمع:", result)
```

خروجی:

سلام فاطمه

جمع: 12

۳. فراخوانی توابع خاص از ماژول

می‌توان فقط یک تابع خاص را از ماژول وارد کرد:

```
from my_module import add

print(add(10, 15))
```

در این حالت، نیاز نیست نام ماژول را قبل از تابع بنویسیم.

۴. استفاده از alias برای ماژول

می‌توانیم به ماژول یک نام کوتاه بدهیم:

```
import my_module as mm

print(mm.multiply(3, 4))
```

۵. اضافه کردن متغیر به ماژول

ماژول می‌تواند متغیر هم داشته باشد:

```
# my_module.py
pi = 3.14159

def circle_area(r):
    return pi * r * r
```

فراخوانی:

```
import my_module

print("مساحت دایره:", my_module.circle_area(5))
```

۶. ماژول شامل کلاس

ماژول می‌تواند کلاس‌ها را هم شامل شود:

```
# my_module.py

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"نام: {self.name}, سن: {self.age}")
```

فراخوانی:

```
from my_module import Person
```

```
p = Person("14", "امیرعلی")
p.info()
```

۷. ماژول با توابع پیشرفته

می‌توان توابعی با پارامترهای اختیاری و چند مقدار بازگشتی ایجاد کرد:

```
# my_module.py

def calculate(a, b):
    sum_ = a + b
    diff = a - b
    prod = a * b
    return sum_, diff, prod
```

۸. استفاده از `"__name__ == '__main__':"`

گاهی می‌خواهیم ماژول هم قابلیت اجرا به صورت مستقل داشته باشد و هم قابلیت وارد شدن به دیگر برنامه‌ها.

```
# my_module.py

def greet(name):
    print("سلام", name)

if __name__ == "__main__":
    # این بخش فقط وقتی فایل مستقیماً اجرا شود، اجرا می‌شود
    greet("دنیا")
```

اگر این ماژول وارد فایل دیگری شود، بخش داخل `__main__` اجرا نمی‌شود.

۹. ساخت ماژول با چند فایل

فرض کنید می‌خواهیم ماژولی با چند زیر ماژول بسازیم:

```
my_package/  
    __init__.py  
    math_ops.py  
    string_ops.py
```

- `math_ops.py` شامل توابع ریاضی
- `string_ops.py` شامل توابع رشته‌ای
- `__init__.py` برای معرفی پکیج

مثال `math_ops.py`:

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

مثال `string_ops.py`:

```
def greet(name):  
    return f"سلام {name}!"
```

۱۰. فراخوانی از پکیج

```
from my_package import math_ops, string_ops

print(math_ops.add(5, 7))
print(string_ops.greet("زهرا"))
```

۱۱. نکات مهم در طراحی ماژول شخصی

۱. نام فایل و نام ماژول باید سازگار باشد و از فاصله یا کاراکترهای خاص استفاده نشود.
۲. از تابع و کلاس‌های کوچک و تخصصی استفاده کنید.
۳. کامنت گذاری برای هر تابع و کلاس مهم است.
۴. برای استفاده مجدد و قابل فهم بودن توابع، بهتر است مستند سازی **Docstring** داشته باشند:

```
def add(a, b):
    """
    تابع جمع دو عدد
    :param a: عدد اول
    :param b: عدد دوم
    :return: جمع a و b
    """
    return a + b
```

۱۲. ماژول با توابع بازگشتی و شرطی

```
# my_module.py

def factorial(n):
    """ به صورت بازگشتی n محاسبه فاکتوریل عدد """
    if n == 0 or n == 1:
```



```
        return 1
    else:
        return n * factorial(n - 1)
```

۱۳. نصب ماژول شخصی در مسیرهای دیگر

می‌توان فایل ماژول را در مسیر **PYTHONPATH** قرار داد تا از هر برنامه‌ای قابل استفاده باشد. مثلاً در ویندوز:

```
C:\Users\YourName\PythonModules
```

و سپس در برنامه:

```
import sys
sys.path.append("C:\\Users\\YourName\\PythonModules")
import my_module
```

۱۴. مثال عملی: ماژول مدیریت دانش‌آموز

```
# student_module.py

students = []

def add_student(name, age):
    students.append({"name": name, "age": age})

def list_students():
    for s in students:
        print(s["name"], "-", s["age"])
```

فراخوانی:

```
import student_module as sm
```

```
sm.add_student("10", "فاطمه")
```

```
sm.add_student("14", "امیرعلی")
```

```
sm.list_students()
```

۱۵. نتیجه‌گیری و نکات کلیدی

- ماژول شخصی باعث کاهش تکرار کد می‌شود.
- برنامه قابل مدیریت و گسترش می‌شود.
- استفاده از ماژول‌ها باعث خوانایی و نظم برنامه می‌شود.
- می‌توان ماژول‌ها را بین پروژه‌ها به اشتراک گذاشت.
- استفاده از `__name__ == "__main__"` باعث کنترل بهتر اجرا می‌شود.
- **پکیج‌ها (Package)** امکان سازمان‌دهی ماژول‌ها در چند فایل را فراهم می‌کنند.

درس هجدهم

فایل ها یکی از مهم ترین ابزارها برای ذخیره سازی داده ها در برنامه نویسی هستند. با استفاده از فایل ها می توان اطلاعات را بین برنامه ها یا اجراهای مختلف یک برنامه ذخیره و بازیابی کرد.

۱. انواع فایل ها

۱. متنی (Text Files)

- شامل کاراکترهای متنی است و با پسوندهایی مثل `.txt`, `.csv`, `.py` ذخیره می شود.
- محتوا معمولاً خواندنی و قابل ویرایش با برنامه های ویرایشگر متن است.

۲. باینری (Binary Files)

- شامل داده های غیرمتنی مثل تصاویر، صدا، و فایل های `.exe`.
- با پسوندهایی مثل `.jpg`, `.png`, `.mp3` ذخیره می شود.

در این آموزش تمرکز روی فایل های متنی است.

۲. باز کردن فایل

در پایتون برای کار با فایل ها از تابع `open()` استفاده می کنیم:

```
file = open("example.txt", mode)
```

- `example.txt` → نام فایل
- `mode` → حالت باز کردن فایل

حالت های رایج:

توضیح	حالت
فقط خواندن (Read)	'r'
نوشتن (Write)، اگر فایل وجود نداشته باشد ایجاد می‌شود	'w'
افزودن به انتهای فایل (Append)	'a'
ایجاد فایل جدید، اگر فایل موجود باشد خطا می‌دهد	'x'
خواندن باینری	'rb'
نوشتن باینری	'wb'
خواندن و نوشتن	'r+'

۳. نوشتن در فایل

```
file = open("data.txt", "w") # باز کردن فایل برای نوشتن
file.write("سلام دنیا!\n") # نوشتن متن
file.write("این یک خط دیگر است\n")
file.close() # بستن فایل
```

توجه: همیشه بعد از اتمام کار فایل را **ببندید** تا داده‌ها ذخیره شوند.

۴. خواندن فایل

```
file = open("data.txt", "r")
content = file.read() # خواندن کل محتوا
```

```
print(content)
file.close()
```

- `read()` → کل محتوا را به صورت یک رشته برمی‌گرداند
- `readline()` → یک خط فایل را می‌خواند
- `readlines()` → تمام خطوط را به صورت **لیست** برمی‌گرداند

مثال با `readline()`

```
file = open("data.txt", "r")
line1 = file.readline()
line2 = file.readline()
print(line1, line2)
file.close()
```

مثال با `readlines()`

```
file = open("data.txt", "r")
lines = file.readlines()
for line in lines:
    print(line.strip()) # حذف کاراکترهای اضافی \n
file.close()
```

۵. استفاده از `with` برای مدیریت فایل

به جای باز و بستن دستی، می‌توان از `with` استفاده کرد:

```
with open("data.txt", "r") as file:
```

```
content = file.read()
print(content)
```

- مزیت: فایل بعد از خروج از بلاک خودکار بسته می‌شود
 - دیگر نیازی به `file.close()` نیست
-

۶. افزودن به فایل (Append)

```
with open("data.txt", "a") as file:
    file.write("خط جدید اضافه شد\n")
```

- 'a' محتوا را پاک نمی‌کند و به انتهای فایل اضافه می‌کند
-

۷. خواندن خط به خط با حلقه

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

- مناسب برای فایل‌های بزرگ که نمی‌خواهیم کل محتوا را یکجا بخوانیم
-

۸. نوشتن چند خط با یک لیست

```
lines = ["خط اول\n", "خط دوم\n", "خط سوم\n"]
with open("data.txt", "w") as file:
    file.writelines(lines)
```

- `writelines()` لیست رشته‌ها را بدون اضافه کردن خودکار `\n` می‌نویسد

۹. مدیریت استثناءها

برای جلوگیری از خطا در باز کردن فایل‌ها:

```
try:
    with open("missing.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("فایل وجود ندارد")
```

۱۰. خواندن بخشی از فایل

```
with open("data.txt", "r") as file:
    part = file.read(10) # خواندن ۱۰ کاراکتر اول
    print(part)
```

۱۱. حرکت در فایل با `seek()` و `tell()`

- `tell()` → موقعیت فعلی اشاره‌گر فایل را برمی‌گرداند
- `seek(pos)` → اشاره‌گر را به موقعیت مشخصی منتقل می‌کند

```
with open("data.txt", "r") as file:
    print(file.read(5)) # خواندن ۵ کاراکتر اول
    print(file.tell()) # موقعیت فعلی
    file.seek(0)       # بازگشت به ابتدا
    print(file.read(5))
```

۱۲. خواندن و نوشتن همزمان (+r)

```
with open("data.txt", "r+") as file:
    content = file.read()
    print(content)
    file.write("\nخط جدید اضافه شد")
```

۱۳. مثال عملی: شمارش تعداد کلمات در فایل

```
with open("data.txt", "r") as file:
    content = file.read()
    words = content.split()
    print("تعداد کلمات:", len(words))
```

۱۴. مثال عملی: فیلتر کردن خطوط خاص

```
with open("data.txt", "r") as file:
    lines = file.readlines()

with open("filtered.txt", "w") as file:
    for line in lines:
        if "سلام" in line:
            file.write(line)
```

- این برنامه خطوط شامل کلمه "سلام" را در فایل جدید ذخیره می‌کند
-

۱۵. خواندن فایل CSV ساده

```
with open("data.csv", "r") as file:
    for line in file:
        row = line.strip().split(",")
        print(row)
```

- هر خط فایل به یک لیست رشته‌ها تبدیل می‌شود

۱۶. نوشتن فایل CSV ساده

```
data = [ ["نام", "سن"], ["فاطمه", "10"], ["امیرعلی", "14"] ]

with open("data.csv", "w") as file:
    for row in data:
        file.write(",".join(row) + "\n")
```

۱۷. استفاده از ماژول csv

```
import csv

# نوشتن CSV
with open("data.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["نام", "سن"])
    writer.writerow(["فاطمه", "10"])
    writer.writerow(["امیرعلی", "14"])

# خواندن CSV
```

```
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

۱۸. مثال عملی: ذخیره و خواندن داده‌های برنامه

فرض کنید می‌خواهیم داده‌های دانش‌آموزان را ذخیره و بعداً بازیابی کنیم:

```
students = [{"name": "فاطمه", "age": 10}, {"name": "امیرعلی", "age": 14}]
```

ذخیره

```
with open("students.txt", "w") as file:
    for s in students:
        file.write(f"{s['name']},{s['age']}\n")
```

خواندن

```
loaded_students = []
with open("students.txt", "r") as file:
    for line in file:
        name, age = line.strip().split(",")
        loaded_students.append({"name": name, "age": int(age)})

print(loaded_students)
```

۱۹. نکات پیشرفته

- همیشه از `with` استفاده کنید تا فایل خودکار بسته شود.

- هنگام نوشتن، مراقب پاک شدن محتوا باشید ('w' پاک می‌کند).
 - از 'a' برای افزودن به انتهای فایل استفاده کنید.
 - برای فایل‌های باینری از حالت‌های 'rb', 'wb' استفاده کنید.
 - بررسی خطاها با try-except ضروری است.
 - استفاده از ماژول‌های استاندارد مانند csv, json و pickle برای داده‌های پیچیده توصیه می‌شود.
-

۲۰. جمع‌بندی

- ✓ فایل‌ها به برنامه‌ها امکان ذخیره و بازیابی اطلاعات را می‌دهند.
- ✓ حالت‌های مختلف 'r', 'a', 'w', '+r' کاربردهای متفاوت دارند.
- ✓ استفاده از with باعث مدیریت خودکار فایل می‌شود.
- ✓ ماژول‌های استاندارد مانند csv و json کار با داده‌های پیچیده را ساده می‌کنند.
- ✓ خواندن و نوشتن فایل‌ها در پایتون بسیار ساده، ولی قوی و کاربردی است.

پروژہ اول

پروژه اول

دفترچه یادداشت متنی ساده

دفترچه یادداشت متنی (Text Notebook) یکی از ساده‌ترین پروژه‌ها برای یادگیری کار با فایل‌ها در پایتون است.

در این پروژه می‌توان:

- یادداشت جدید اضافه کرد
- یادداشت‌ها را مشاهده کرد
- یادداشت‌ها را جستجو کرد
- یادداشت‌ها را حذف یا ویرایش کرد

۱. ساختار فایل‌ها

تمام یادداشت‌ها در یک فایل متنی ذخیره می‌شوند، مثلاً:

`notes.txt`

هر خط فایل یک یادداشت را نشان می‌دهد.

۲. باز و بسته کردن فایل‌ها

برای نوشتن یا خواندن از دستور `with open` استفاده می‌کنیم:

```
with open("notes.txt", "a") as file:  
    file.write("این یک یادداشت جدید است.\n")
```

- حالت `'a'` → افزودن به انتهای فایل بدون پاک کردن محتوا

۳. خواندن تمام یادداشت‌ها

```
with open("notes.txt", "r") as file:
    notes = file.readlines()
    for idx, note in enumerate(notes, start=1):
        print(f"{idx}. {note.strip()}")
```

- `strip()` → حذف کاراکترهای اضافی مانند `\n`
-

۴. اضافه کردن یادداشت جدید

```
def add_note():
    note = input("متن یادداشت را وارد کنید: ")
    with open("notes.txt", "a") as file:
        file.write(note + "\n")
    print("یادداشت اضافه شد")
```

۵. نمایش یادداشت‌ها

```
def show_notes():
    try:
        with open("notes.txt", "r") as file:
            notes = file.readlines()
            if not notes:
                print("هیچ یادداشتی موجود نیست.")
            for idx, note in enumerate(notes, start=1):
                print(f"{idx}. {note.strip()}")
    except FileNotFoundError:
        print("فایل یادداشت موجود نیست.")
```

۶. جستجوی یادداشت‌ها

```
def search_notes():
    keyword = input("کلمه کلیدی را وارد کنید: ")
    with open("notes.txt", "r") as file:
        notes = file.readlines()
        found = False
        for idx, note in enumerate(notes, start=1):
            if keyword in note:
                print(f"{idx}. {note.strip()}")
                found = True
        if not found:
            print("هیچ یادداشتی یافت نشد.")
```

۷. حذف یک یادداشت

```
def delete_note():
    show_notes()
    try:
        index = int(input("شماره یادداشتی که می‌خواهید حذف کنید را وارد کنید: "))
        with open("notes.txt", "r") as file:
            notes = file.readlines()
        if 1 <= index <= len(notes):
            removed = notes.pop(index - 1)
            with open("notes.txt", "w") as file:
                file.writelines(notes)
            print(f"!حذف شد '{removed.strip()}' یادداشت")
        else:
```



```
        print("شماره وارد شده معتبر نیست.")
except ValueError:
    print("عدد معتبر وارد کنید.")
```

۸. ویرایش یک یادداشت

```
def edit_note():
    show_notes()
    try:
        index = int(input("شماره یادداشتی که می‌خواهید ویرایش کنید را وارد کنید: "))
        with open("notes.txt", "r") as file:
            notes = file.readlines()
        if 1 <= index <= len(notes):
            new_text = input("متن جدید را وارد کنید: ")
            notes[index - 1] = new_text + "\n"
            with open("notes.txt", "w") as file:
                file.writelines(notes)
            print("یادداشت ویرایش شد")
        else:
            print("شماره وارد شده معتبر نیست.")
    except ValueError:
        print("عدد معتبر وارد کنید.")
```

۹. منوی اصلی برنامه

```
def main_menu():
    while True:
        print("\n--- دفترچه یادداشت ---")
```

```
print("1. نمایش یادداشت‌ها")
print("2. اضافه کردن یادداشت")
print("3. جستجوی یادداشت")
print("4. حذف یادداشت")
print("5. ویرایش یادداشت")
print("6. خروج")

choice = input("یک گزینه را انتخاب کنید: ")

if choice == "1":
    show_notes()
elif choice == "2":
    add_note()
elif choice == "3":
    search_notes()
elif choice == "4":
    delete_note()
elif choice == "5":
    edit_note()
elif choice == "6":
    print("خروج از برنامه...")
    break
else:
    print("گزینه معتبر نیست. دوباره تلاش کنید.")
```

۱. اجرای برنامه

```
if __name__ == "__main__":
    main_menu()
```

۱۱. نکات آموزشی

1. از حالت 'a' برای افزودن یادداشت جدید استفاده شد تا محتوا پاک نشود.
2. برای خواندن و نمایش یادداشت‌ها از 'r' استفاده شد.
3. برای ویرایش و حذف، ابتدا کل محتوا خوانده شد، تغییرات اعمال شد و سپس با 'w' نوشته شد.
4. استفاده از `with open` → مدیریت خودکار فایل و جلوگیری از خطا.
5. استفاده از `strip()` برای حذف کاراکترهای اضافی مثل `n\`.
6. استفاده از `try-except` برای جلوگیری از خطا در ورودی یا نبود فایل.
7. برنامه با منو طراحی شده تا کاربر پسند باشد.

۱۲. نکات پیشرفته

- می‌توان تاریخ و ساعت یادداشت را با ماژول `datetime` اضافه کرد:

```
from datetime import datetime

note = input("یادداشت جدید: ")
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open("notes.txt", "a") as file:
    file.write(f"[{timestamp}] {note}\n")
```

- امکان مرتب‌سازی یادداشت‌ها بر اساس تاریخ یا محتوا وجود دارد.
 - می‌توان از فایل `CSV` برای ذخیره‌سازی ساختار یافته استفاده کرد.
-

۱۳. جمع‌بندی

- با این پروژه ساده، خواندن، نوشتن، ویرایش و حذف فایل‌ها را یاد گرفتیم.
- نحوه استفاده از حالت‌های 'r', 'w', 'a' و '+r' تمرین شد.
- مدیریت خطا، حلقه و منوی تعاملی را آموزش دیدیم.
- می‌توان پروژه را گسترش داد و به یک دفترچه یادداشت پیشرفته تبدیل کرد.

کد کامل پروژه دفترچه یادداشت

```
from datetime import datetime

def add_note():
    note = input("متن یادداشت را وارد کنید: ")
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("notes.txt", "a") as file:
        file.write(f"[{timestamp}] {note}\n")
    print("یادداشت اضافه شد")

def show_notes():
    try:
        with open("notes.txt", "r") as file:
            notes = file.readlines()
            if not notes:
                print("هیچ یادداشتی موجود نیست")
            for idx, note in enumerate(notes, start=1):
                print(f"{idx}. {note.strip()}")
    except FileNotFoundError:
        print("فایل یادداشت موجود نیست")
```

```
def search_notes():
    keyword = input("کلمه کلیدی را وارد کنید: ")
    try:
        with open("notes.txt", "r") as file:
            notes = file.readlines()
            found = False
            for idx, note in enumerate(notes, start=1):
                if keyword in note:
                    print(f"{idx}. {note.strip()}")
                    found = True
            if not found:
                print("هیچ یادداشتی یافت نشد.")
    except FileNotFoundError:
        print("فایل یادداشت موجود نیست.")

def delete_note():
    show_notes()
    try:
        index = int(input("شماره یادداشتی که می‌خواهید حذف کنید را وارد کنید: "))
        with open("notes.txt", "r") as file:
            notes = file.readlines()
        if 1 <= index <= len(notes):
            removed = notes.pop(index - 1)
            with open("notes.txt", "w") as file:
                file.writelines(notes)
            print(f"!حذف شد '{removed.strip()}' یادداشت")
        else:
            print("شماره وارد شده معتبر نیست.")
    except ValueError:
        print("عدد معتبر وارد کنید.")
```

```

except FileNotFoundError:
    print("فایل یادداشت موجود نیست.")

def edit_note():
    show_notes()
    try:
        index = int(input("شماره یادداشتی که می‌خواهید ویرایش کنید را وارد کنید:
"))
        with open("notes.txt", "r") as file:
            notes = file.readlines()
        if 1 <= index <= len(notes):
            new_text = input("متن جدید را وارد کنید: ")
            timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            notes[index - 1] = f"[{timestamp}] {new_text}\n"
            with open("notes.txt", "w") as file:
                file.writelines(notes)
            print("یادداشت ویرایش شد")
        else:
            print("شماره وارد شده معتبر نیست.")
    except ValueError:
        print("عدد معتبر وارد کنید.")
    except FileNotFoundError:
        print("فایل یادداشت موجود نیست.")

def main_menu():
    while True:
        print("\n--- دفترچه یادداشت ---")
        print("1. نمایش یادداشت‌ها")
        print("2. اضافه کردن یادداشت")
        print("3. جستجوی یادداشت")
        print("4. حذف یادداشت")

```

```
print("5. ویرایش یادداشت")
print("6. خروج")

choice = input("یک گزینه را انتخاب کنید: ")

if choice == "1":
    show_notes()
elif choice == "2":
    add_note()
elif choice == "3":
    search_notes()
elif choice == "4":
    delete_note()
elif choice == "5":
    edit_note()
elif choice == "6":
    print("خروج از برنامه...")
    break
else:
    print("گزینه معتبر نیست. دوباره تلاش کنید.")

if __name__ == "__main__":
    main_menu()
```

پروژہ دوم

پروژه دوم

ماشین حساب ساده

ماشین حساب ساده یکی از پروژه‌های کلاسیک برای تمرین توابع، ورودی/خروجی و منوهای تعاملی است. در این پروژه می‌توان:

- جمع، تفریق، ضرب و تقسیم انجام داد
- محاسبه توان و ریشه مربع
- محاسبه باقی‌مانده و تقسیم صحیح
- چند عملیات مختلف را به صورت تعاملی انجام داد

۱. طراحی توابع پایه

هر عملیات ریاضی را می‌توان در یک تابع جداگانه قرار داد:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b != 0:  
        return a / b  
    else:  
        return "خطا: تقسیم بر صفر"
```

۲. توابع پیشرفته

```
def power(a, b):  
    return a ** b  
  
def square_root(a):  
    if a >= 0:  
        return a ** 0.5  
    else:  
        return "خطا: عدد منفی"  
  
def mod(a, b):  
    return a % b  
  
def int_division(a, b):  
    if b != 0:  
        return a // b  
    else:  
        return "خطا: تقسیم بر صفر"
```

۳. تابع گرفتن ورودی عدد

برای جلوگیری از خطاهای ورودی:

```
def get_number(prompt):  
    while True:  
        try:  
            num = float(input(prompt))  
            return num  
        except ValueError:
```

```
print("لطفاً عدد معتبر وارد کنید.")
```

۴. نمایش منوی عملیات

```
def show_menu():  
    print("\n--- ماشین حساب ساده ---")  
    print("1. جمع")  
    print("2. تفریق")  
    print("3. ضرب")  
    print("4. تقسیم")  
    print("5. توان")  
    print("6. ریشه مربع")  
    print("7. باقی مانده (mod)")  
    print("8. تقسیم صحیح")  
    print("9. خروج")
```

۵. تابع اجرای عملیات

```
def perform_operation(choice):  
    if choice in ["1", "2", "3", "4", "5", "7", "8"]:  
        a = get_number("عدد اول: ")  
        b = get_number("عدد دوم: ")  
  
        if choice == "1":  
            print("نتیجه:", add(a, b))  
        elif choice == "2":  
            print("نتیجه:", subtract(a, b))  
        elif choice == "3":  
            print("نتیجه:", multiply(a, b))
```

```
elif choice == "4":
    print("نتیجه:", divide(a, b))
elif choice == "5":
    print("نتیجه:", power(a, b))
elif choice == "7":
    print("نتیجه:", mod(a, b))
elif choice == "8":
    print("نتیجه:", int_division(a, b))

elif choice == "6":
    a = get_number("عدد موردنظر برای ریشه مربع: ")
    print("نتیجه:", square_root(a))

elif choice == "9":
    print("خروج از برنامه")
    return False
else:
    print("گزینه معتبر نیست. دوباره تلاش کنید")
return True
```

۶. حلقه اصلی برنامه

```
def main():
    while True:
        show_menu()
        choice = input("یک گزینه را انتخاب کنید: ")
        if not perform_operation(choice):
            break
```

۷. اجرای برنامه

```
if __name__ == "__main__":  
    main()
```

۸. نکات آموزشی

1. استفاده از توابع جداگانه باعث نظم و قابلیت استفاده مجدد کد می‌شود.
 2. توابع ورودی امن (مثل `get_number`) خطاهای کاربر را مدیریت می‌کنند.
 3. منو و حلقه اصلی تجربه کاربری تعاملی ایجاد می‌کند.
 4. کنترل خطاها مثل تقسیم بر صفر یا عدد منفی در ریشه مربع ضروری است.
 5. برنامه می‌تواند به راحتی گسترش یابد و عملیات جدید اضافه شود.
-

۹. نکات پیشرفته

- می‌توان از ماژول `math` برای عملیات دقیق‌تر استفاده کرد:

```
import math  
  
def square_root(a):  
    if a >= 0:  
        return math.sqrt(a)  
    else:  
        return "خطا: عدد منفی"
```

- می‌توان نتایج را در فایل ذخیره کرد تا تاریخچه عملیات داشته باشیم:

```
def log_result(result):  
    with open("calculator_log.txt", "a") as file:  
        file.write(str(result) + "\n")
```

- امکان افزودن **عملیات علمی** مثل لگاریتم و سینوس و کسینوس با استفاده از `math` وجود دارد.

۱۰. مثال کامل با ذخیره تاریخچه

```
import math  
from datetime import datetime  
  
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b != 0:  
        return a / b  
    else:  
        return "خطا: تقسیم بر صفر"  
  
def power(a, b):
```

```
        return a ** b

def square_root(a):
    if a >= 0:
        return math.sqrt(a)
    else:
        return "خطا: عدد منفی"

def mod(a, b):
    return a % b

def int_division(a, b):
    if b != 0:
        return a // b
    else:
        return "خطا: تقسیم بر صفر"

def get_number(prompt):
    while True:
        try:
            num = float(input(prompt))
            return num
        except ValueError:
            print("لطفاً عدد معتبر وارد کنید.")

def log_result(result):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("calculator_log.txt", "a") as file:
        file.write(f"[{timestamp}] {result}\n")

def show_menu():
```

```
print("\n--- ماشین حساب ساده ---")
print("1. جمع")
print("2. تفریق")
print("3. ضرب")
print("4. تقسیم")
print("5. توان")
print("6. ریشه مربع")
print("7. باقی مانده (mod)")
print("8. تقسیم صحیح")
print("9. خروج")
```

```
def perform_operation(choice):
    if choice in ["1", "2", "3", "4", "5", "7", "8"]:
        a = get_number("عدد اول: ")
        b = get_number("عدد دوم: ")
        if choice == "1":
            result = add(a, b)
        elif choice == "2":
            result = subtract(a, b)
        elif choice == "3":
            result = multiply(a, b)
        elif choice == "4":
            result = divide(a, b)
        elif choice == "5":
            result = power(a, b)
        elif choice == "7":
            result = mod(a, b)
        elif choice == "8":
            result = int_division(a, b)
        print("نتیجه:", result)
        log_result(result)
```



```
elif choice == "6":
    a = get_number("عدد موردنظر برای ریشه مربع: ")
    result = square_root(a)
    print("نتیجه:", result)
    log_result(result)

elif choice == "9":
    print("خروج از برنامه.")
    return False
else:
    print("گزینه معتبر نیست. دوباره تلاش کنید.")
return True

def main():
    while True:
        show_menu()
        choice = input("یک گزینه را انتخاب کنید: ")
        if not perform_operation(choice):
            break

if __name__ == "__main__":
    main()
```

۱۱. جمع‌بندی

- این پروژه عملیات پایه ریاضی را پوشش می‌دهد.
- نحوه استفاده از توابع، حلقه‌ها، شرط‌ها و ورودی کاربر آموزش داده شد.

- ذخیره نتایج در فایل باعث شد با **خواندن و نوشتن فایل‌ها** ترکیب شود.
- برنامه آماده گسترش به ماشین حساب علمی یا پیشرفته است.

پروژه سوم

پروژه سوم

بازی حدس عدد

بازی حدس عدد یکی از پروژه‌های محبوب برای تمرین ورودی کاربر، شرط‌ها، حلقه‌ها و مدیریت فایل است. در این پروژه می‌توان:

- عدد تصادفی توسط کامپیوتر انتخاب شود
- کاربر حدس بزند و برنامه راهنمایی کند
- تعداد تلاش‌ها شمرده شود
- رکوردها ذخیره و نمایش داده شوند

۱. وارد کردن ماژول‌های لازم

برای تولید عدد تصادفی و ذخیره رکوردها از فایل:

```
import random
from datetime import datetime
```

۲. تولید عدد تصادفی

```
def generate_number(low=1, high=100):
    return random.randint(low, high)
```

- `randint(low, high)` عددی بین کمترین و بیشترین مقدار تولید می‌کند

۳. گرفتن حدس از کاربر

```
def get_guess():
```

```
while True:
    try:
        guess = int(input("حدس خود را وارد کنید: "))
        return guess
    except ValueError:
        print("لطفاً یک عدد صحیح وارد کنید")
```

۴. بررسی حدس

```
def check_guess(number, guess):
    if guess < number:
        print("بیشتر تلاش کنید")
        return False
    elif guess > number:
        print("کمتر تلاش کنید")
        return False
    else:
        print("تبریک! حدس درست است")
        return True
```

۵. ذخیره رکوردها در فایل

```
def save_score(player_name, attempts):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("guess_number_scores.txt", "a") as file:
        file.write(f"{timestamp} - {player_name}: {attempts} تلاش\n")
```

۶. نمایش رکوردها

```
def show_scores():
    try:
        with open("guess_number_scores.txt", "r") as file:
            scores = file.readlines()
            if not scores:
                print("هیچ رکوردی موجود نیست.")
            else:
                print("\n--- رکوردها ---")
                for score in scores:
                    print(score.strip())
    except FileNotFoundError:
        print("هیچ رکوردی هنوز ثبت نشده است.")
```

۷. بازی اصلی

```
def play_game():
    print("\n--- بازی حدس عدد ---")
    player_name = input("نام شما: ")
    number = generate_number()
    attempts = 0

    while True:
        guess = get_guess()
        attempts += 1
        if check_guess(number, guess):
            print(f"تعداد تلاش‌ها: {attempts}")
            save_score(player_name, attempts)
            break
```

۸. منوی بازی

```
def main_menu():  
    while True:  
        print("\n--- منوی بازی حدس عدد ---")  
        print("1. شروع بازی جدید")  
        print("2. نمایش رکوردها")  
        print("3. خروج")  
  
        choice = input("یک گزینه را انتخاب کنید: ")  
  
        if choice == "1":  
            play_game()  
        elif choice == "2":  
            show_scores()  
        elif choice == "3":  
            print("خروج از بازی...")  
            break  
        else:  
            print("گزینه معتبر نیست. دوباره تلاش کنید.")
```

۹. اجرای برنامه

```
if __name__ == "__main__":  
    main_menu()
```

۱. نکات آموزشی

1. استفاده از **توابع جداگانه** باعث نظم و خوانایی کد شد.
 2. با حلقه و شرطها، **راهنمایی کاربر** در حدسها انجام شد.
 3. تعداد تلاشها شمارش شد و ذخیره در فایل باعث شد **رکوردها حفظ شوند**.
 4. از ماژولهای استاندارد **random** و **datetime** استفاده شد.
 5. استفاده از **try-except** → جلوگیری از خطا در ورودی کاربر.
-

۱۱. نکات پیشرفته

- می‌توان محدوده عدد را به صورت **پویا** از کاربر گرفت:

```
low = int(input("حداقل عدد: "))
high = int(input("حداکثر عدد: "))
number = generate_number(low, high)
```

- می‌توان **حداکثر تلاش** تعیین کرد و بازی را محدود کرد:

```
max_attempts = 10
if attempts >= max_attempts:
    print("تلاش‌ها به پایان رسید! عدد درست", number)
    break
```

- امکان افزودن **آمار کلی** مثل بهترین رکورد یا میانگین تلاش‌ها با خواندن فایل وجود دارد.
-

۱۲. ذخیره و نمایش بهترین رکورد

```
def show_best_score():
    try:
        with open("guess_number_scores.txt", "r") as file:
            scores = file.readlines()
            if not scores:
                print("هیچ رکوردی موجود نیست.")
                return
            best_attempts = min([int(s.split(": ")[1].split())[0]) for
s in scores])
            print(f"تلاش: {best_attempts} بهترین رکورد")
    except FileNotFoundError:
        print("فایل رکورد موجود نیست.")
```

۱۳. نسخه کامل با بهترین رکورد

```
import random
from datetime import datetime

def generate_number(low=1, high=100):
    return random.randint(low, high)

def get_guess():
    while True:
        try:
            guess = int(input("حدس خود را وارد کنید: "))
            return guess
        except ValueError:
            print("لطفاً یک عدد صحیح وارد کنید")
```

```
def check_guess(number, guess):
    if guess < number:
        print("بیشتر تلاش کنید")
        return False
    elif guess > number:
        print("کمتر تلاش کنید")
        return False
    else:
        print("تبریک! حدس درست است")
        return True

def save_score(player_name, attempts):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("guess_number_scores.txt", "a") as file:
        file.write(f"{timestamp} - {player_name}: {attempts} تلاش\n")

def show_scores():
    try:
        with open("guess_number_scores.txt", "r") as file:
            scores = file.readlines()
            if not scores:
                print("هیچ رکوردی موجود نیست")
            else:
                print("\n--- رکوردها ---")
                for score in scores:
                    print(score.strip())
    except FileNotFoundError:
        print("هیچ رکوردی هنوز ثبت نشده است")

def show_best_score():
```

```

try:
    with open("guess_number_scores.txt", "r") as file:
        scores = file.readlines()
        if not scores:
            print("هیچ رکوردی موجود نیست.")
            return
        best_attempts = min([int(s.split(": ")[1].split())[0]) for
s in scores])
        print(f"تلاش: {best_attempts} بهترین رکورد")
except FileNotFoundError:
    print("فایل رکورد موجود نیست.")

def play_game():
    print("\n--- بازی حدس عدد ---")
    player_name = input("نام شما: ")
    number = generate_number()
    attempts = 0

    while True:
        guess = get_guess()
        attempts += 1
        if check_guess(number, guess):
            print(f"تعداد تلاش‌ها: {attempts}")
            save_score(player_name, attempts)
            show_best_score()
            break

def main_menu():
    while True:
        print("\n--- منوی بازی حدس عدد ---")
        print("شروع بازی جدید 1.")

```

```
print("2. نمایش رکوردها")
print("3. نمایش بهترین رکورد")
print("4. خروج")

choice = input("یک گزینه را انتخاب کنید: ")

if choice == "1":
    play_game()
elif choice == "2":
    show_scores()
elif choice == "3":
    show_best_score()
elif choice == "4":
    print("خروج از بازی...")
    break
else:
    print("گزینه معتبر نیست. دوباره تلاش کنید.")

if __name__ == "__main__":
    main_menu()
```

۱۴. جمع‌بندی

- بازی حدس عدد آموزش توابع، حلقه‌ها و شرط‌ها را تمرین می‌کند.
- استفاده از فایل باعث شد ذخیره رکورد و تاریخچه بازی فراهم شود.
- برنامه قابل گسترش به بازی پیشرفته با محدوده‌های پویا و تعداد تلاش محدود است.
- استفاده از ماژول‌های `random` و `datetime` کاربرد واقعی ماژول‌ها را نشان می‌دهد.

درس نوزدهم

درس نوزدهم

معرفی کتابخانه‌های محبوب پایتون (مثل Turtle و Pygame)

پایتون یکی از زبان‌های برنامه‌نویسی محبوب و قدرتمند است که به دلیل سادگی و خوانایی، در بسیاری از زمینه‌ها مورد استفاده قرار می‌گیرد. یکی از دلایل قدرت پایتون، **کتابخانه‌ها (Libraries)** و **ماژول‌ها (Modules)** است که امکانات زیادی را به برنامه‌نویس می‌دهند.

۱. کتابخانه چیست؟

- **کتابخانه (Library)** مجموعه‌ای از کدهای آماده است که برای انجام وظایف خاص طراحی شده‌اند.
- به جای نوشتن همه چیز از صفر، می‌توانیم از کتابخانه‌ها استفاده کنیم.
- مثال: انجام محاسبات علمی، رسم نمودار، ساخت بازی، طراحی رابط کاربری و غیره.

۲. تفاوت ماژول و کتابخانه

ماژول (Module)	کتابخانه (Library)
یک فایل پایتون	مجموعه‌ای از ماژول‌ها
شامل توابع و کلاس‌ها	شامل چندین ماژول با توابع و کلاس‌های مرتبط
مثال: <code>math</code>	مثال: <code>NumPy</code>

۳. کتابخانه Turtle

Turtle یکی از کتابخانه‌های استاندارد پایتون است که برای **رسم گرافیک ساده** و آموزش برنامه‌نویسی به کار می‌رود.

ویژگی‌ها

- رسم خطوط و اشکال هندسی
- کنترل حرکت "لاک پشت" (Cursor) در صفحه
- مناسب برای آموزش و سرگرمی

مثال ساده

```
import turtle

t = turtle.Turtle()
t.forward(100) # حرکت به جلو ۱۰۰ پیکسل
t.left(90)     # چرخش ۹۰ درجه به چپ
t.forward(100)
turtle.done()
```

نکات مهم Turtle

- می‌توان از **حلقه‌ها** برای رسم اشکال پیچیده استفاده کرد.
- از **رنگ‌ها و پر کردن (fill)** می‌توان گرافیک جذاب‌تر ایجاد کرد.
- مناسب آموزش کودکان و مبتدیان است.

۴. کتابخانه Pygame

Pygame برای ساخت بازی‌های دو بعدی در پایتون استفاده می‌شود.

ویژگی‌ها

- کنترل صفحه نمایش، تصاویر و صدا
- مدیریت ورودی کاربر (کیبورد و موس)

- امکان ساخت بازی‌های تعاملی و سرگرم‌کننده

مثال ساده

```
import pygame

pygame.init()
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("نمونه بازی")
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

pygame.quit()
```

نکات مهم Pygame

- مناسب برای پروژه‌های آموزشی و بازی‌های کوچک
- یادگیری مفاهیم **Loop** و **Event Handling** را تقویت می‌کند
- می‌توان صدا، انیمیشن و برخورد اشیاء را مدیریت کرد

۵. کتابخانه Tkinter

Tkinter برای ساخت رابط کاربری گرافیکی (GUI) در پایتون کاربرد دارد.

ویژگی‌ها

- ساخت دکمه، لیست، منو و پنجره
- نمایش پیام و ورودی کاربر
- مناسب برای برنامه‌های رسمی و کاربردی

مثال ساده

```
import tkinter as tk

root = tk.Tk()
root.title("پنجره ساده")
label = tk.Label(root, text="!سلام پایتون")
label.pack()
root.mainloop()
```

نکات مهم Tkinter

- کتابخانه رسمی پایتون است
- برای پروژه‌های اپلیکیشن دسکتاپ مناسب است
- یادگیری آن باعث می‌شود برنامه‌های کاربرپسند بسازیم

۶. کتابخانه Matplotlib

Matplotlib برای رسم نمودار و تصویرسازی داده‌ها استفاده می‌شود.

ویژگی‌ها

- رسم نمودار خطی، میله‌ای، دایره‌ای و پراکندگی
- مناسب تحلیل داده و علم داده

- یک ابزار مهم برای آمار و گرافیک علمی

مثال ساده

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.title("نمودار نمونه")
plt.xlabel("محور X")
plt.ylabel("محور Y")
plt.show()
```

نکات مهم Matplotlib

- می‌توان نمودارها را ذخیره و شخصی‌سازی کرد
- با کتابخانه NumPy و Pandas ترکیب قدرتمندی دارد
- مناسب برای تحلیل داده و آموزش علوم داده

۷. کتابخانه NumPy

NumPy برای محاسبات عددی و علمی و کار با آرایه‌ها استفاده می‌شود.

ویژگی‌ها

- آرایه‌های چند بعدی (ndarray)
- عملیات ریاضی سریع و بهینه

- پایه‌ای برای علم داده و یادگیری ماشین

مثال ساده

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)  # جمع عنصر به عنصر
print(a * b)  # ضرب عنصر به عنصر
```

نکات مهم NumPy

- سرعت بسیار بالاتر از لیست‌های پایتون برای عملیات ریاضی
- پایه اکثر کتابخانه‌های علم داده مانند TensorFlow و Pandas
- مناسب تحلیل آماری و علمی

۸. کتابخانه Pandas

Pandas برای مدیریت و تحلیل داده‌ها استفاده می‌شود.

ویژگی‌ها

- کار با **DataFrame** (جدول داده)
- خواندن و نوشتن CSV و Excel
- تحلیل و فیلتر داده‌ها

مثال ساده

```
import pandas as pd

data = {'نام': ['علی', 'فاطمه'], 'سن': [20, 25]}
df = pd.DataFrame(data)
print(df)
```

نکات مهم Pandas

- مناسب برای تحلیل داده‌های جدولی
- توابع قدرتمند برای مرتب‌سازی، فیلتر و گروه‌بندی داده‌ها
- ترکیب عالی با NumPy و Matplotlib

۹. کتابخانه Requests

Requests برای ارسال درخواست HTTP و ارتباط با وب استفاده می‌شود.

ویژگی‌ها

- ارسال GET و POST
- دریافت داده از API
- مدیریت پاسخ‌ها و JSON

مثال ساده

```
import requests

response = requests.get("https://jsonplaceholder.typicode.com/todos/1")
print(response.json())
```

نکات مهم Requests

- مناسب پروژه‌های وب و API
- یادگیری آن برای دریافت و ارسال داده آنلاین ضروری است
- ساده‌تر از کتابخانه استاندارد `urllib`

۱. کتابخانه BeautifulSoup

BeautifulSoup برای استخراج داده از صفحات وب (Web Scraping) کاربرد دارد.

ویژگی‌ها

- پردازش HTML و XML
- استخراج متن، لینک و اطلاعات خاص
- مناسب برای پروژه‌های تحلیل وب و داده‌کاوی

مثال ساده

```
from bs4 import BeautifulSoup

html = "<html><body><h1>سلام</h1></body></html>"
soup = BeautifulSoup(html, "html.parser")
print(soup.h1.text) # خروجی: سلام
```

۱۱. کتابخانه OpenCV

OpenCV برای پردازش تصویر و ویدئو استفاده می‌شود.

ویژگی‌ها

- شناسایی چهره، اشیا و حرکات
- پردازش و فیلتر تصاویر
- کاربرد در بینایی ماشین و هوش مصنوعی

مثال ساده

```
import cv2

img = cv2.imread("image.jpg")
cv2.imshow("تصویر", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

۱۲. کتابخانه SciPy

SciPy برای محاسبات علمی و مهندسی استفاده می‌شود و بر پایه NumPy است.

ویژگی‌ها

- الگوریتم‌های بهینه‌سازی، انتگرال، حل معادلات
- تحلیل داده‌های علمی و مهندسی
- مناسب مهندسان و محققان

۱۳. کتابخانه SymPy

SymPy برای ریاضیات نمادین کاربرد دارد.

ویژگی‌ها

- حل معادلات و انتگرال‌ها به صورت نمادین
- نمایش ریاضیاتی فرمول‌ها
- مناسب برای آموزش ریاضی و علوم پایه

مثال ساده

```
from sympy import symbols, solve

x = symbols('x')
eq = x**2 - 4
solution = solve(eq, x)
print(solution) # خروجی: [-2, 2]
```

۱۴. کتابخانه SciKit-Learn

Scikit-Learn برای یادگیری ماشین استفاده می‌شود.

ویژگی‌ها

- الگوریتم‌های طبقه‌بندی، خوشه‌بندی و پیش‌بینی
- ابزارهای آماده برای پروژه‌های **ML**
- کار آسان با **NumPy** و **Pandas**

۱۵. کتابخانه TensorFlow و PyTorch

- برای هوش مصنوعی و شبکه‌های عصبی
- محاسبات سریع GPU

- یادگیری عمیق و پیش‌بینی

مثال ساده با TensorFlow

```
import tensorflow as tf

print(tf.constant("!سلام پایتون"))
```

۱۶. کتابخانه Seaborn

- برای تصویرسازی داده‌ها
- مناسب نمودارهای آماری
- ترکیب عالی با **Matplotlib** و **Pandas**

۱۷. کتابخانه Plotly

- نمودارهای تعاملی و وب
- مناسب داشبوردهای داده و برنامه‌های تحت وب
- گرافیک جذاب و پویا

۱۸. کتابخانه Kivy

- ساخت اپلیکیشن موبایل و دسکتاپ
- رابط کاربری پیشرفته و چندلمسی

- مناسب برنامه‌نویسی حرفه‌ای GUI

۱۹. کتابخانه PyInstaller

- تبدیل کد پایتون به فایل اجرایی (EXE)
- مناسب انتشار نرم‌افزار بدون نیاز به نصب پایتون

۲۰. جمع‌بندی

- پایتون دارای هزاران کتابخانه برای علم داده، بازی، وب، GUI، پردازش تصویر و هوش مصنوعی است
- با استفاده از کتابخانه‌ها می‌توان کارهای پیچیده را با چند خط کد انجام داد
- انتخاب کتابخانه مناسب بسته به نوع پروژه و هدف است
- ترکیب کتابخانه‌ها مثل **NumPy + Pandas + Matplotlib** بسیار رایج و قدرتمند است

نکات پایانی

نکات پایانی

نکات و مسیرهای پیشرفت بعدی

پایان یک کتاب، همیشه آغاز یک مسیر جدید است. هر فردی که قدم در دنیای برنامه‌نویسی می‌گذارد، با یادگیری مبانی شروع می‌کند، اما مسیر رشد او به **تمرین، پروژه‌محوری و یادگیری مداوم** بستگی دارد.

۱. اهمیت یادگیری مداوم

یادگیری در برنامه‌نویسی هیچ وقت متوقف نمی‌شود. حتی حرفه‌ای‌ترین برنامه‌نویسان همیشه در حال **کشف ابزارها، کتابخانه‌ها و تکنیک‌های جدید** هستند.

- هر روز، فناوری‌های جدید معرفی می‌شوند
- زبان‌ها و کتابخانه‌ها به‌روز می‌شوند
- مهارت‌های جدید در حوزه‌های مختلف مثل وب، هوش مصنوعی، داده‌کاوی و گیمینگ شکل می‌گیرد

توصیه عملی

- روزانه حداقل ۳۰ دقیقه برنامه‌نویسی کنید
- پروژه‌های کوچک را امتحان کنید و کم‌کم پروژه‌های بزرگ‌تر بسازید
- همیشه **کد دیگران را مطالعه کنید** تا سبک‌های مختلف برنامه‌نویسی را بیاموزید

۲. تمرین و ساخت پروژه‌های عملی

هیچ چیزی به اندازه **ساخت پروژه واقعی** به شما کمک نمی‌کند. پروژه‌ها باعث می‌شوند:

- مفاهیم نظری را به کار عملی تبدیل کنید
- با مشکلات واقعی کدنویسی روبه‌رو شوید

- اعتماد به نفس برنامه‌نویسی شما افزایش یابد

مسیر پیشنهادی پروژه‌ها

1. **پروژه‌های ساده:** ماشین حساب، دفترچه یادداشت، بازی حدس عدد
2. **پروژه‌های متوسط:** بازی‌های ساده با Pygame، تحلیل داده‌های CSV با Pandas
3. **پروژه‌های پیشرفته:** وب اپلیکیشن با Flask یا Django، پردازش تصویر با OpenCV

۳. یادگیری کتابخانه‌ها و ماژول‌های پیشرفته

کتابخانه‌ها ابزارهایی هستند که شما را از نوشتن کد تکراری نجات می‌دهند. مسیر پیشرفت شما شامل:

- یادگیری کتابخانه‌های استاندارد پایتون مثل `os`, `sys`, `datetime`, `math`
- تسلط بر کتابخانه‌های علم داده: NumPy, Pandas, Matplotlib, Seaborn
- یادگیری کتابخانه‌های بازی و گرافیک: Turtle, Pygame
- آشنایی با وب و API: Requests, BeautifulSoup, Flask

توصیه عملی

- برای هر کتابخانه یک پروژه کوچک بسازید
- مستندات رسمی کتابخانه را مطالعه کنید
- مثال‌های آموزشی و کدهای آماده را تمرین کنید

۴. تمرین مهارت‌های حل مسئله

برنامه‌نویسی یعنی حل مسئله. هر چه در حل مسائل منطقی و ریاضی مهارت داشته باشید، برنامه‌نویس بهتری خواهید شد.

- از سایت‌های تمرینی مثل **Codewars** و **LeetCode**، **HackerRank** استفاده کنید
- الگوریتم‌ها و ساختار داده‌ها را یاد بگیرید
- مسئله‌ها را به بخش‌های کوچک تقسیم و به تدریج حل کنید

۵. یادگیری اصول طراحی نرم‌افزار

پس از یادگیری مبانی، بهتر است با اصول طراحی نرم‌افزار آشنا شوید:

- **کدنویسی تمیز (Clean Code):** نوشتن کدی که خوانا، قابل فهم و قابل نگهداری باشد
- **تفکیک وظایف (Modular Programming):** تقسیم کد به توابع و ماژول‌ها
- **مدیریت پروژه:** استفاده از نسخه کنترل (Git و GitHub)

توصیه عملی

- پروژه‌های کوچک خود را روی **GitHub** قرار دهید
- تغییرات کد خود را با commit های معنی‌دار ذخیره کنید
- با استفاده از branch ها، ویژگی‌های جدید را آزمایش کنید

۶. مهارت‌های تحلیل داده و علوم داده

اگر علاقه‌مند به تحلیل داده هستید:

- **Pandas** و **NumPy** را برای مدیریت داده‌ها یاد بگیرید
- با **Seaborn** و **Matplotlib** نمودار و تحلیل تصویری انجام دهید
- با **SciPy** و **Scikit-Learn** وارد دنیای یادگیری ماشین شوید

مسیر پیشرفت

1. تحلیل داده‌های ساده CSV
2. پیش‌بینی با الگوریتم‌های ساده
3. پروژه‌های یادگیری ماشین کوچک

۷. برنامه‌نویسی وب و توسعه نرم‌افزار

اگر علاقه به ساخت وب اپلیکیشن و برنامه‌های کاربردی دارید:

- با **Flask** یا **Django** شروع کنید
- یادگیری HTML، CSS و JavaScript ضروری است
- ساخت وبسایت‌های کوچک، ورود و ثبت‌نام، فروشگاه ساده

توصیه عملی

- یک **وب اپلیکیشن شخصی** بسازید و روی سرور محلی اجرا کنید
- داده‌ها را از فایل یا دیتابیس دریافت و نمایش دهید
- به تدریج قابلیت‌های بیشتر اضافه کنید

۸. یادگیری برنامه‌نویسی شیء‌گرا (OOP)

OOP یا برنامه‌نویسی شیء‌گرا برای ساخت **برنامه‌های بزرگ و قابل توسعه** ضروری است:

- مفاهیم کلاس، شیء، ارث‌بری و چندریختی را یاد بگیرید
 - پروژه‌ها را به کلاس‌های مستقل تقسیم کنید
 - درک این مفاهیم باعث می‌شود **کد سازماندهی شده و قابل نگهداری باشد**
-

۹. یادگیری گرافیک و بازی سازی

برای علاقه‌مندان به بازی:

- **Turtle** برای شروع و آموزش مفاهیم گرافیکی
- **Pygame** برای ساخت بازی‌های دو بعدی
- پروژه‌های کوچک مثل Snake، Pong یا بازی حدس عدد گرافیکی

توصیه عملی

- ابتدا بازی ساده بسازید
- بعد از تسلط، به بازی‌های پیچیده‌تر و با انیمیشن بروید
- صدا، تصویر و برخورد اشیاء را به بازی اضافه کنید

۱۰. هوش مصنوعی و یادگیری ماشین

برای مسیر پیشرفته و علاقه‌مندان به AI:

- **NumPy + Pandas + Matplotlib** پایه‌ای ضروری است
- یادگیری **Scikit-Learn** برای الگوریتم‌های طبقه‌بندی و پیش‌بینی
- پروژه‌های کوچک مانند تشخیص اعداد MNIST یا پیش‌بینی دما

توصیه عملی

- پروژه‌های کوچک بسازید و نتایج را ذخیره کنید
- مدل‌ها را با داده‌های واقعی تمرین دهید
- درک ریاضیاتی الگوریتم‌ها بسیار مهم است

۱۱. تمرین روزانه و چالش

- برنامه‌نویسی مهارتی است که با **تمرین مداوم** رشد می‌کند
- چالش‌های روزانه باعث افزایش سرعت و دقت می‌شود
- همیشه خود را در معرض **مسائل واقعی و پروژه‌های عملی** قرار دهید

۱۲. مشارکت در جامعه برنامه‌نویسی

- عضویت در **گروه‌ها و انجمن‌ها** باعث افزایش دانش و ارتباط با دیگران می‌شود
- مشارکت در **Stack Overflow، GitHub، و Reddit** فرصت یادگیری عملی فراهم می‌کند
- با همکاری، تجربه واقعی توسعه نرم‌افزار کسب می‌کنید

۱۳. مستندات و یادگیری مستقل

- همیشه مستندات رسمی کتابخانه‌ها و زبان را مطالعه کنید
- یادگیری مستقل باعث می‌شود از **آموزش‌ها فراتر بروید**
- مثال‌ها و پروژه‌های آماده را بررسی و تحلیل کنید

۱۴. ثبت پیشرفت شخصی

- نتایج پروژه‌ها و یادگیری خود را **دفترچه یا فایل دیجیتال** ثبت کنید
- اهداف کوتاه‌مدت و بلندمدت تعیین کنید

- بررسی پیشرفت باعث انگیزه و افزایش اعتماد به نفس می‌شود

۱۵. ساختار مسیر پیشرفت

مسیر مقدماتی

1. یادگیری مفاهیم پایه پایتون
2. تمرین پروژه‌های کوچک و ساده
3. مدیریت فایل، توابع، ماژول و کتابخانه‌های استاندارد

مسیر متوسط

1. ساخت پروژه‌های متوسط (دفترچه یادداشت، ماشین حساب، بازی ساده)
2. آشنایی با GUI و گرافیک
3. تحلیل داده و نمودار

مسیر پیشرفته

1. وب اپلیکیشن و برنامه‌های کاربردی
2. پروژه‌های علمی و یادگیری ماشین
3. بازی‌های حرفه‌ای و پردازش تصویر

۱۶. نکات انگیزشی

- هر اشتباه یک درس و فرصت یادگیری است
- برنامه‌نویسی مهارت حل مسئله است، نه فقط نوشتن کد

- صبر و پشتکار، کلید موفقیت است
 - هیچ مسیر یادگیری خطی نیست، گاهی باید بازگشت و اصلاح کنید
-

۱۷. توصیه‌های عملی نهایی

1. روزانه کد بنویسید و پروژه‌های کوچک بسازید
 2. یادگیری کتابخانه‌ها و ابزارها را ادامه دهید
 3. پروژه‌ها را با **GitHub** و **جامعه برنامه‌نویسان** به اشتراک بگذارید
 4. مستندات را مطالعه کنید و مثال‌های عملی اجرا کنید
 5. خودتان را در چالش‌های برنامه‌نویسی و مسابقات آنلاین قرار دهید
-

۱۸. جمع‌بندی مسیر پیشرفت

- مسیر یادگیری شامل مبانی → پروژه‌های عملی → کتابخانه‌ها → مهارت‌های پیشرفته است
- مهم‌ترین نکته تمرین مداوم و پروژه‌محوری است
- با پیشرفت در مسیر، می‌توانید به توسعه نرم‌افزار، هوش مصنوعی، بازی‌سازی یا تحلیل داده‌ها وارد شوید